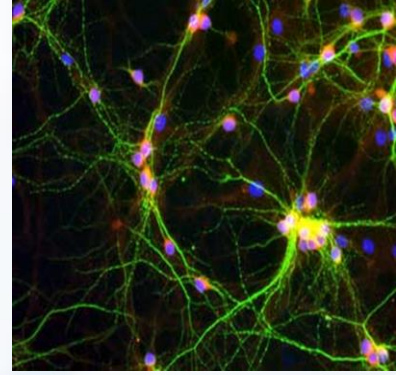


Artificial Neural Networks

Lecture 5



Plutarch: the brain is not a container needs to be filled with knowledge, but a torch needs to be lighted





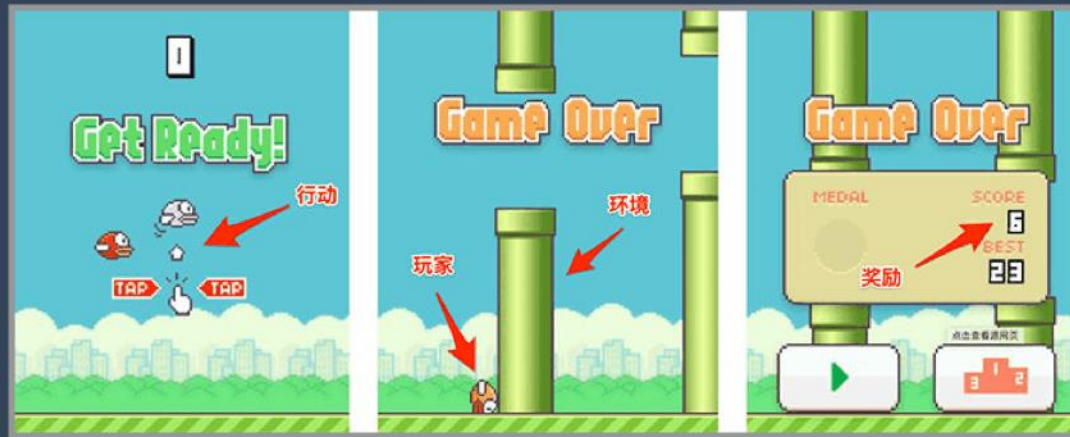
Part I : Reinforcement learning



With respect to the feedback type to learner:

- ❑ **Supervised learning** : Task Driven (Classification)
 - ❑ **Unsupervised learning** : Data Driven (Clustering)
 - ❑ **Reinforcement learning** —
 - Close to human learning.
 - Algorithm learns a policy of how to act in a given environment.
 - Every action has some impact in the environment, and the environment provides rewards that guides the learning algorithm.
-

Reinforcement learning



What is reinforcement learning?

Reinforcement learning (RL) is a way for computers to learn independently by making a series of decisions and learning from the outcomes. Through trial and error, computer programs determine the best actions within a certain context and optimize their performance.

The computer receives positive or negative feedback based on its actions and gradually learns how to complete a task. In other words, RL is about learning the optimal behavior in an environment to obtain maximum reward.



Reinforcement learning

Supervised Learning

■ Step: 1

Teacher: Does picture 1 show a car or a flower?

Learner: A flower.

Teacher: No, it's a car.

Step: 2

Teacher: Does picture 2 show a car or a flower?

Learner: A car.

Teacher: Yes, it's a car.

Step: 3

Reinforcement Learning

Step: 1

World: You are in state 9. Choose action A or C.

Learner: Action A.

World: Your reward is 100.

Step: 2

World: You are in state 32. Choose action B or E.

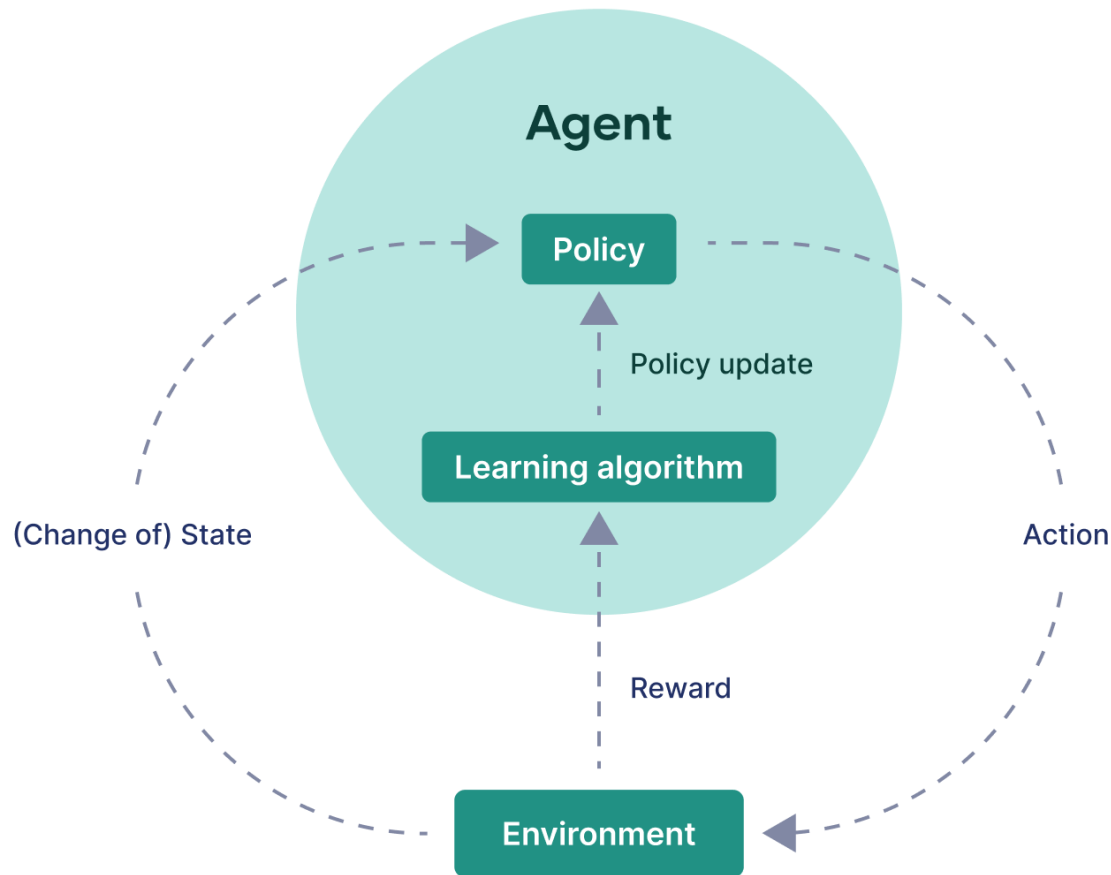
Learner: Action B.

World: Your reward is 50.

Step: 3

Reinforcement learning

The general framework of reinforcement learning



- **Agent** refers to the learner or decision-maker (i.e., the computer program) within the environment. The agent explores the environment and interacts with it.

- **Action** refers to moves taken by an agent within the environment.

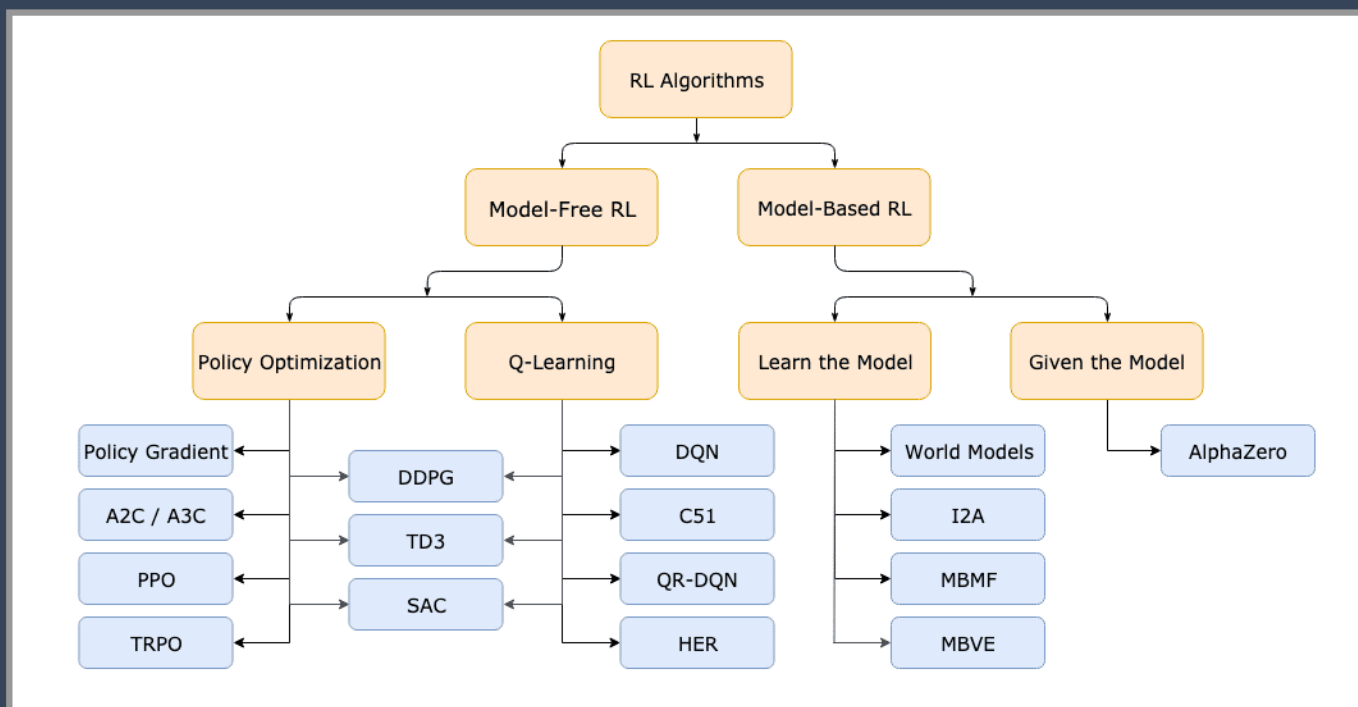
- **State** is the current situation of the agent at a specific time. Each action leads to a change of state.

- **Reward** can be **positive**, to reinforce good behavior, or **negative**, to deter undesirable behavior. It is feedback that the agent receives from the environment for a certain action.

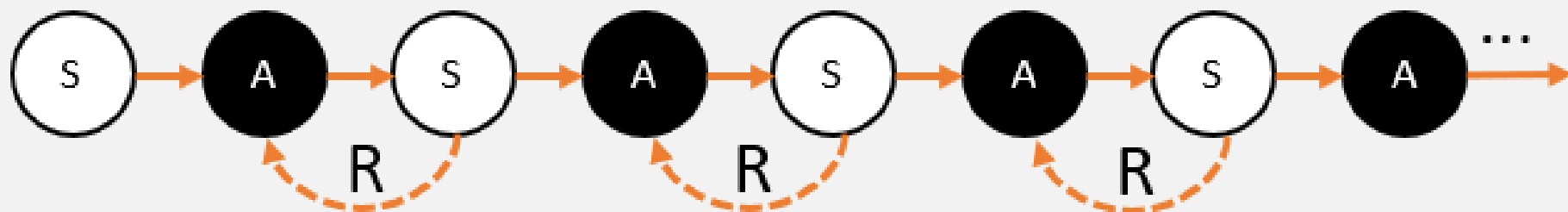


Reinforcement learning

主流的强化学习算法分类



Reinforcement learning



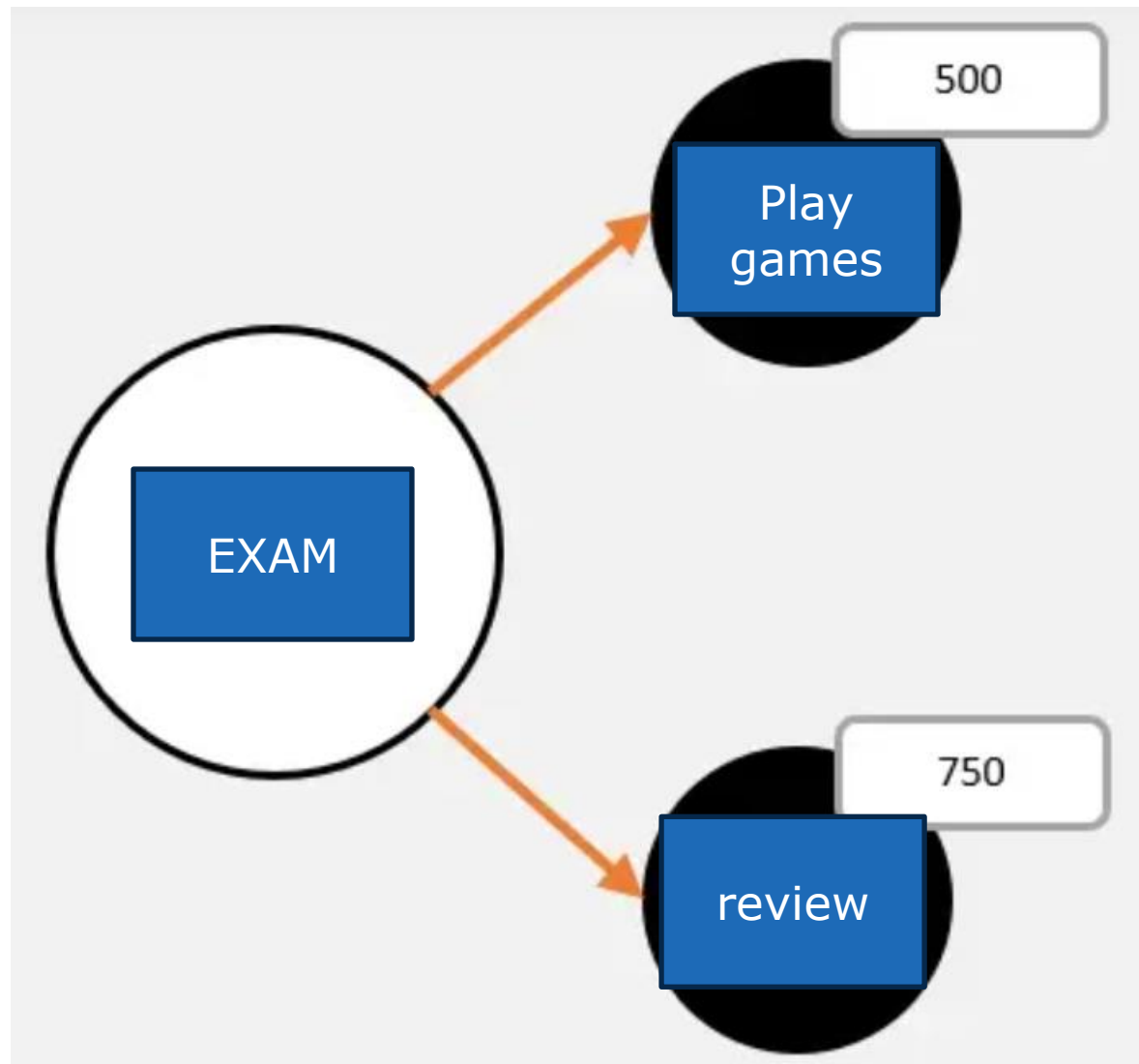
S (state)

A (action)

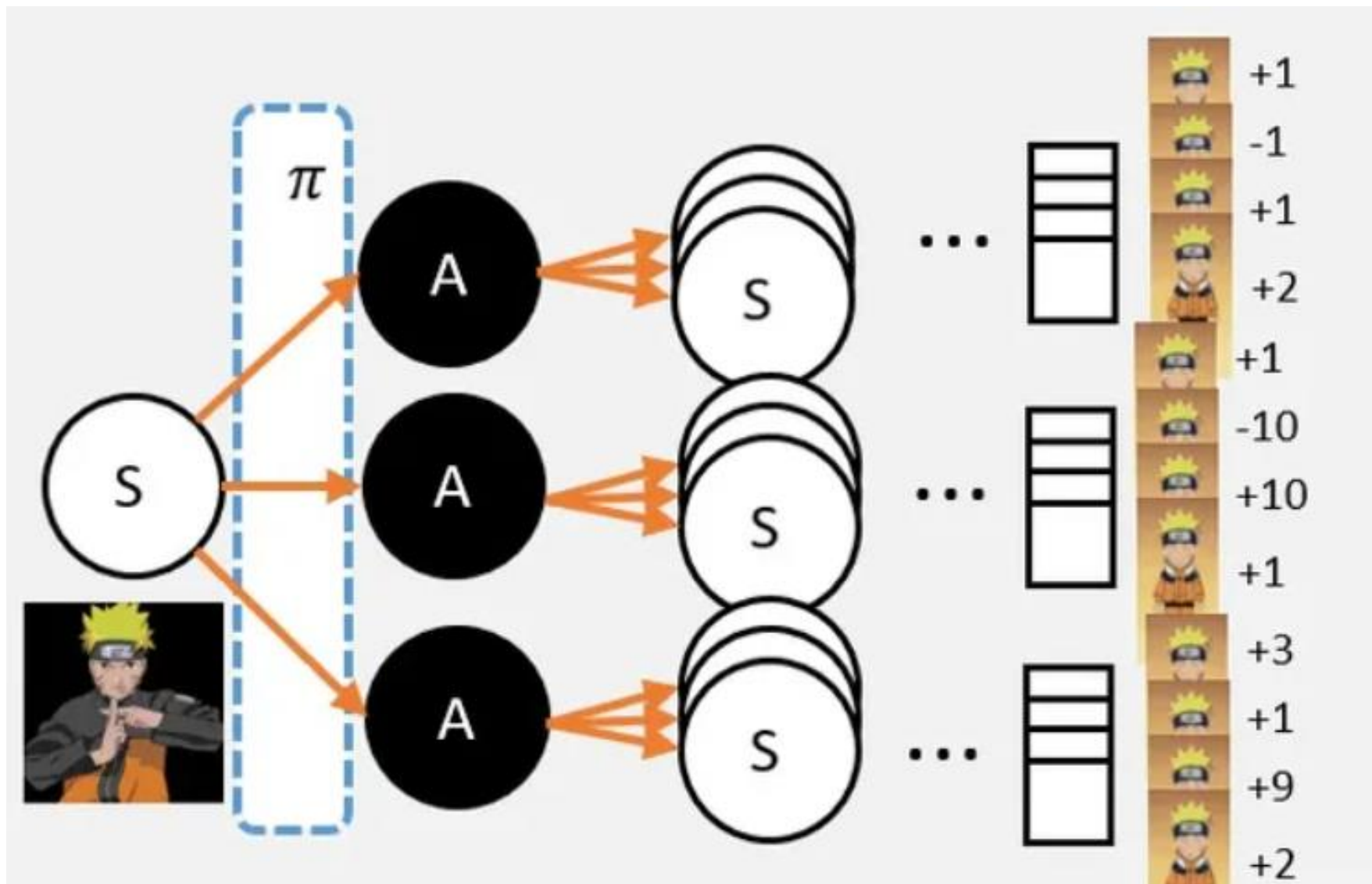
R (reward)



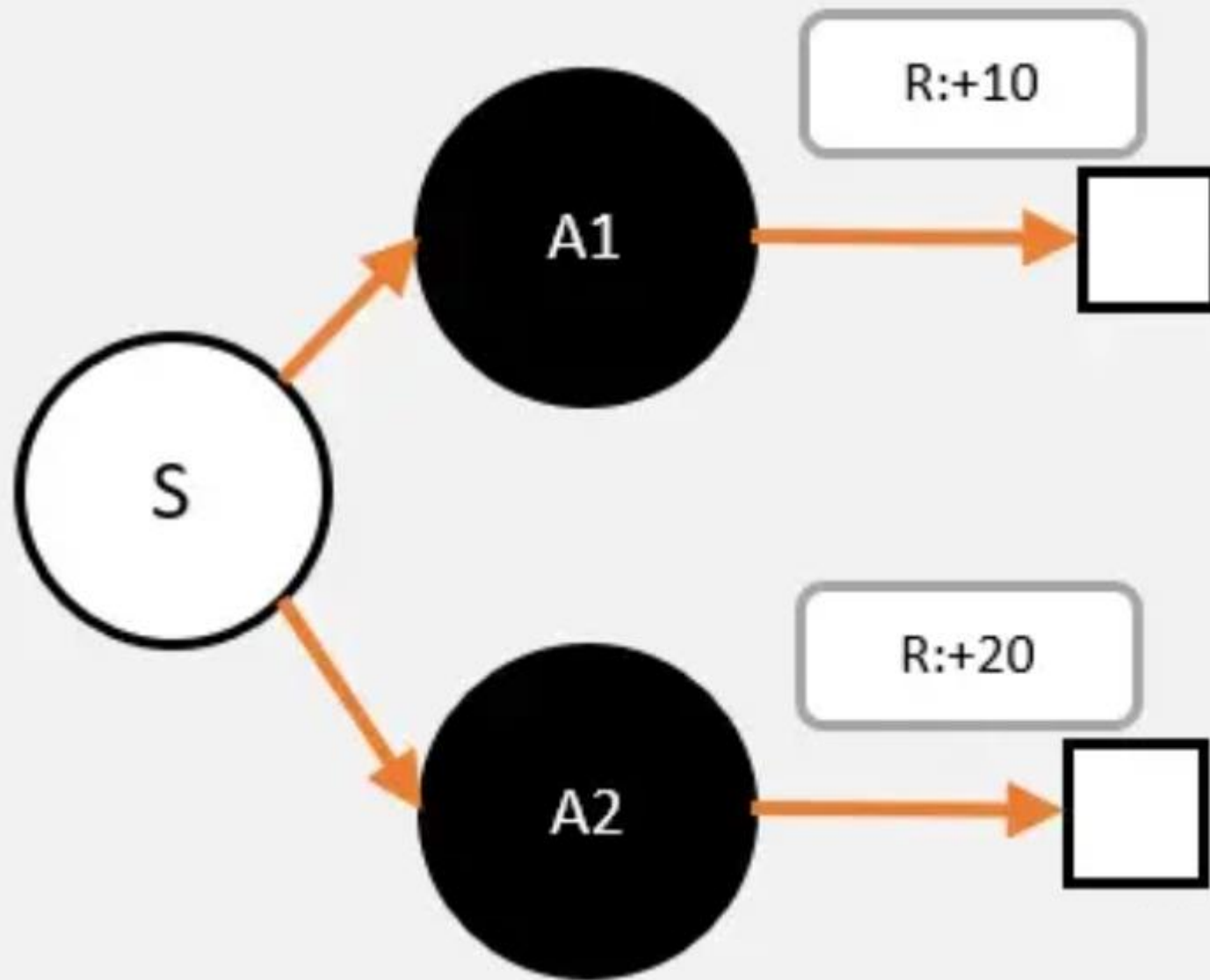
Reinforcement learning



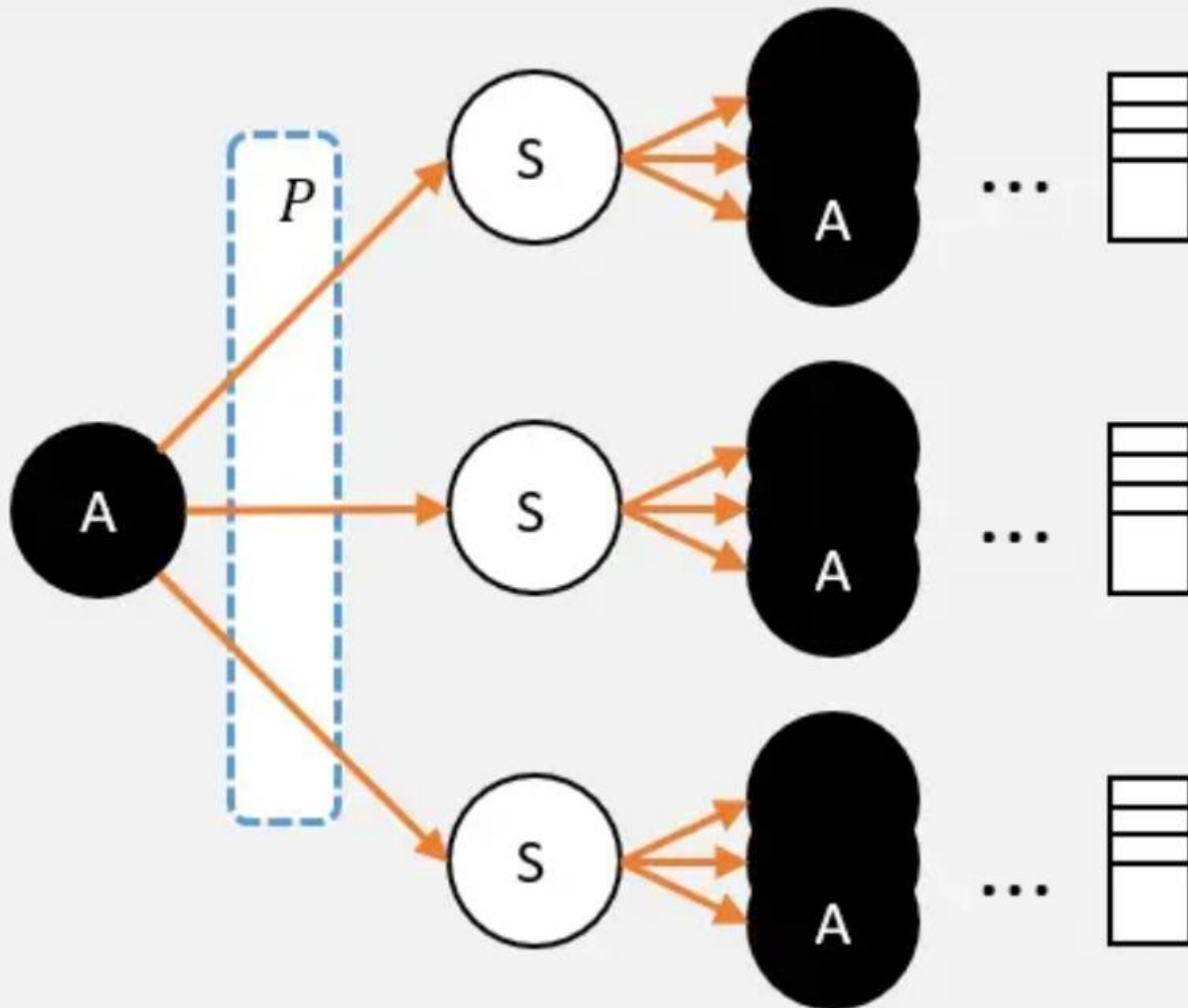
Reinforcement learning



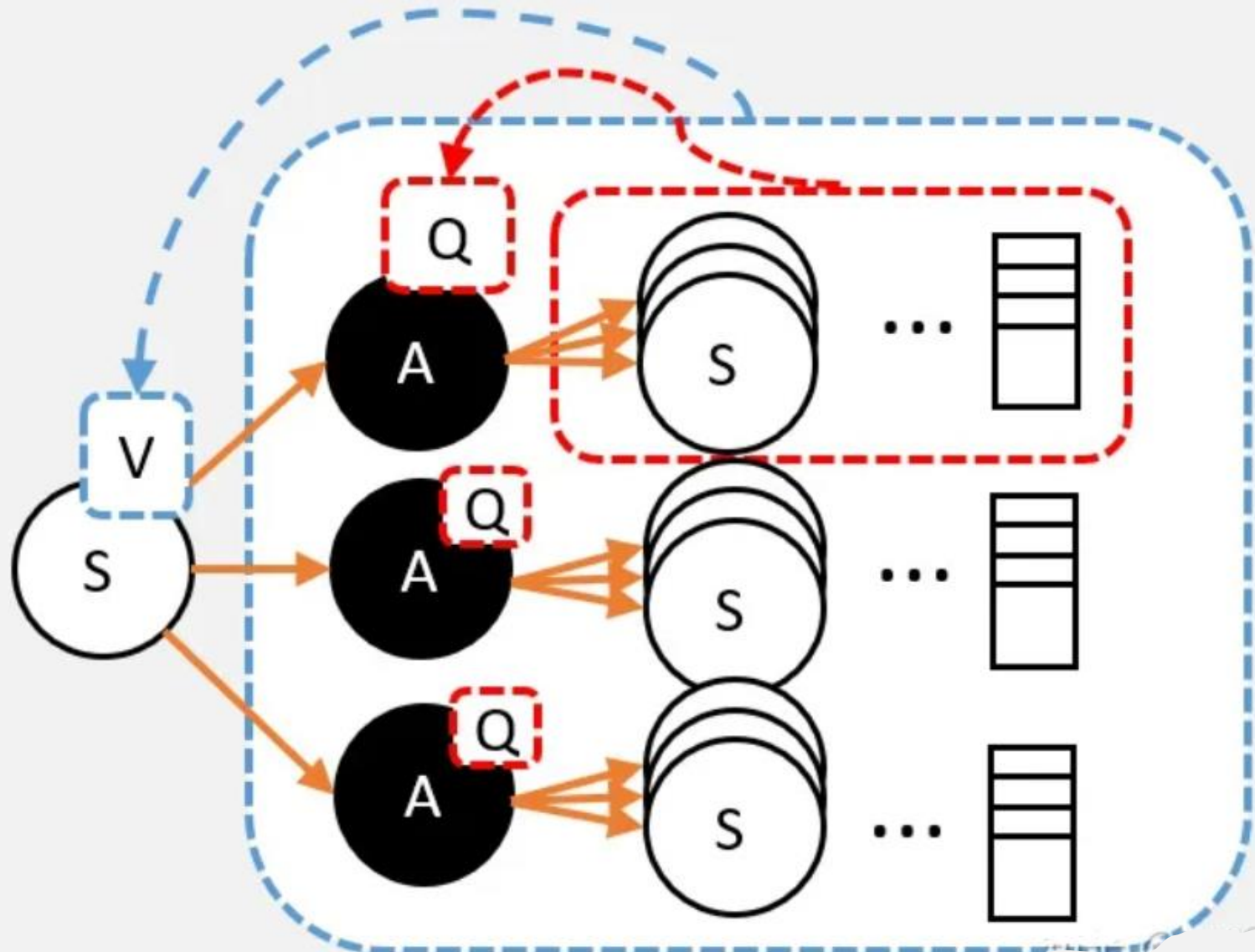
Reinforcement learning



Reinforcement learning



Reinforcement learning

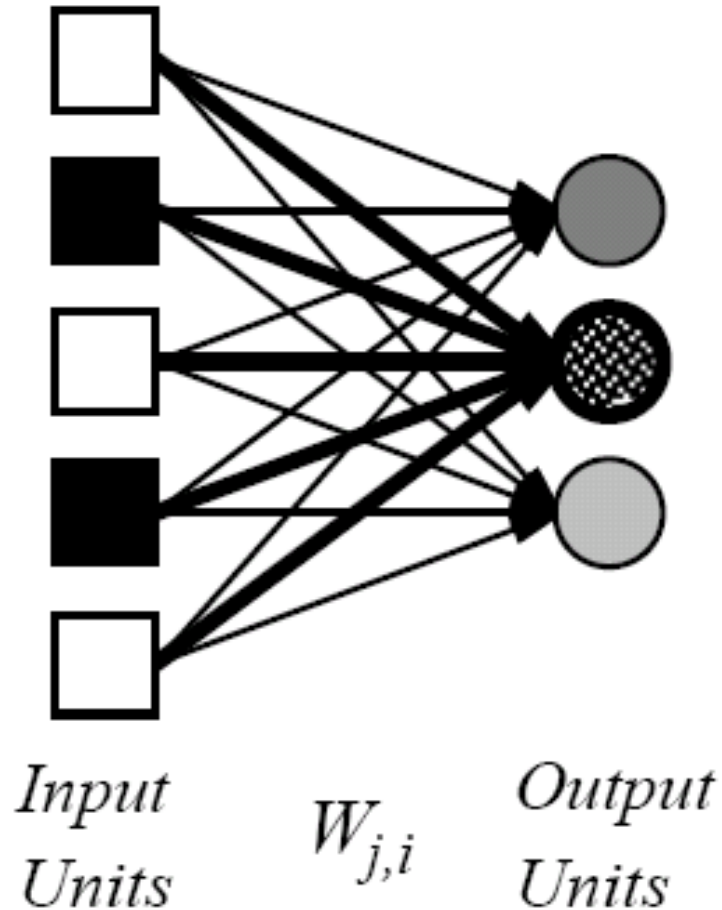




Part II: MLP



2.1 Single-layer Perceptron



- ❖ **Binary Threshold Activation Function**
- ❖ **Output units all operate separately, i.e. no shared weights**

Rosenblatt, 1957

Single-layer Perceptron Learning

- ❖ Adjust weights to **reduce error** on training set
- ❖ Simple weight update rule:

$$\omega_{ij} \leftarrow \omega_{ij} + \eta x_i \underbrace{(d_j - y_j)}_{\delta}$$

desired output

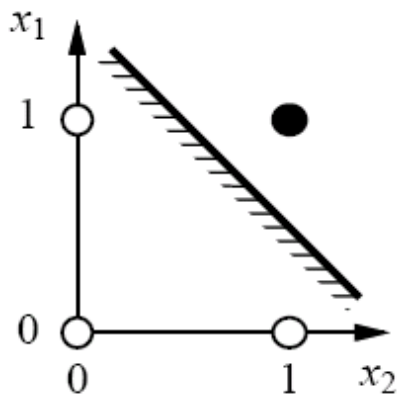
generated output

$$y_j = f\left(\sum_{i=0}^m \omega_{ij} x_i\right)$$

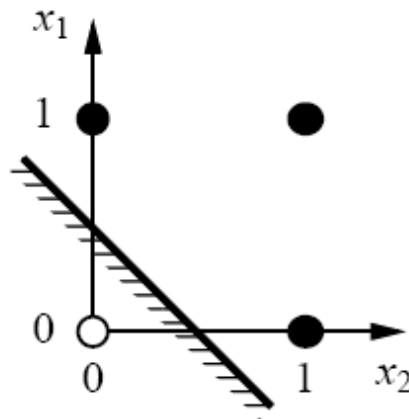


Expressiveness of Single-layer Perceptron

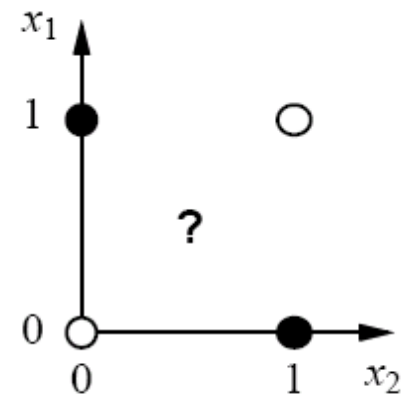
- ❖ Can represent AND, OR, NOT, etc., but not XOR
- ❖ Represents a **linear separator** :



(a) x_1 and x_2



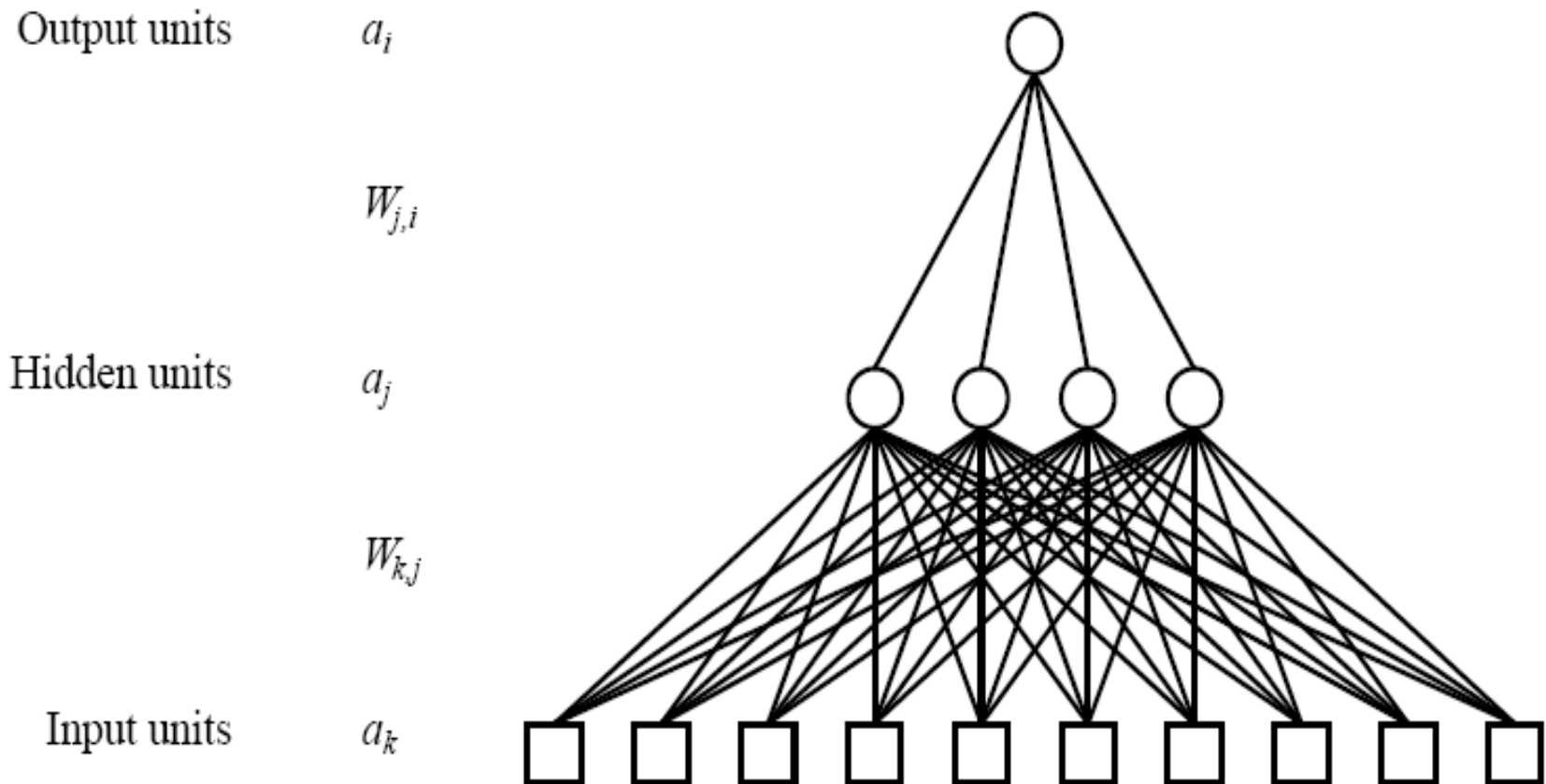
(b) x_1 or x_2



(c) x_1 xor x_2

Minsky & Papert (1969) pricked the neural network balloon

Multi-layer Perceptron (MLP)





Deep Architectures

Deep architectures are composed of multiple levels (usually >3) of non-linear operations, such as neural nets with many hidden layers.



Usefulness of Deep Architectures

- ❖ **Eg., learning feature hierarchies, where features from higher level of the hierarchy are formed by lower level features.**

edges, local shapes, object parts

Low level representation

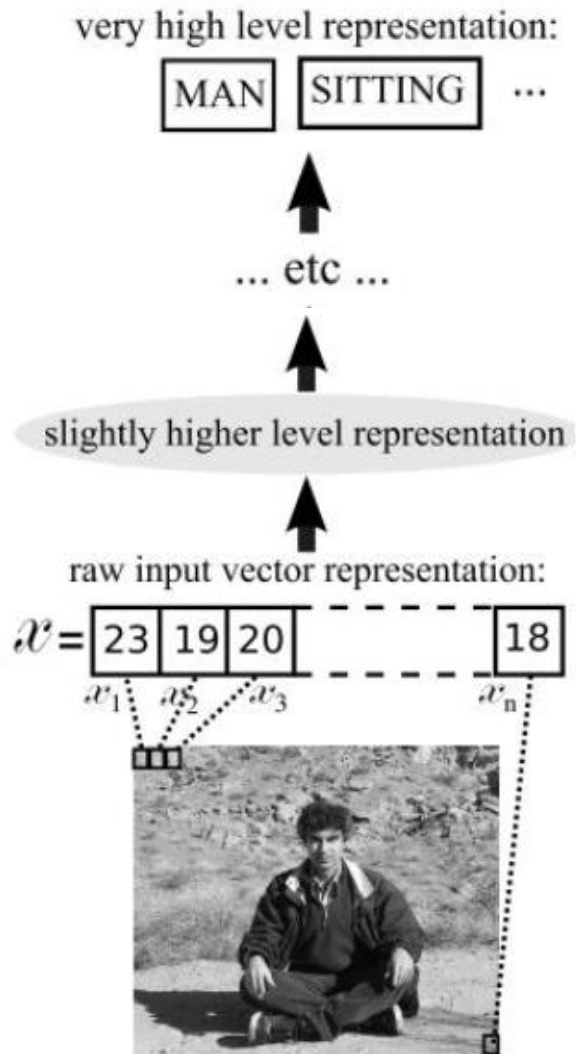


Figure is from Yoshua Bengio

AI:ANN



Advantages of Deep Architectures

- ❖ **Neurobiological Motivation:** The mammal brain is organized in a deep architecture (Serre, Kreiman, Kouh, Cadieu, Knoblich, & Poggio, 2007) (E.g. visual system has 5 to 10 levels)
- ❖ Some functions cannot be efficiently represented (in terms of number of tunable elements) by architectures that are too shallow.
- ❖ Deep architectures might be able to represent some functions otherwise not efficiently representable.





Advantages of Deep Architectures Cont'd

- ❖ **More formally:** Functions that can be compactly represented by a depth k architecture might require an exponential number of computational elements to be represented by a depth $k - 1$ architecture
- ❖ The consequences are
 - **Computational:** We don't need exponentially many elements in the layers
 - **Statistical:** poor generalization may be expected when using an insufficiently deep architecture for representing some functions.



Expressiveness of MLPs

❖ **3 layers: All continuous functions**

❖ **4 layers: all functions**

How to learn the weights???

waiting B-P algorithm until 1986

❖ **More Layers:**

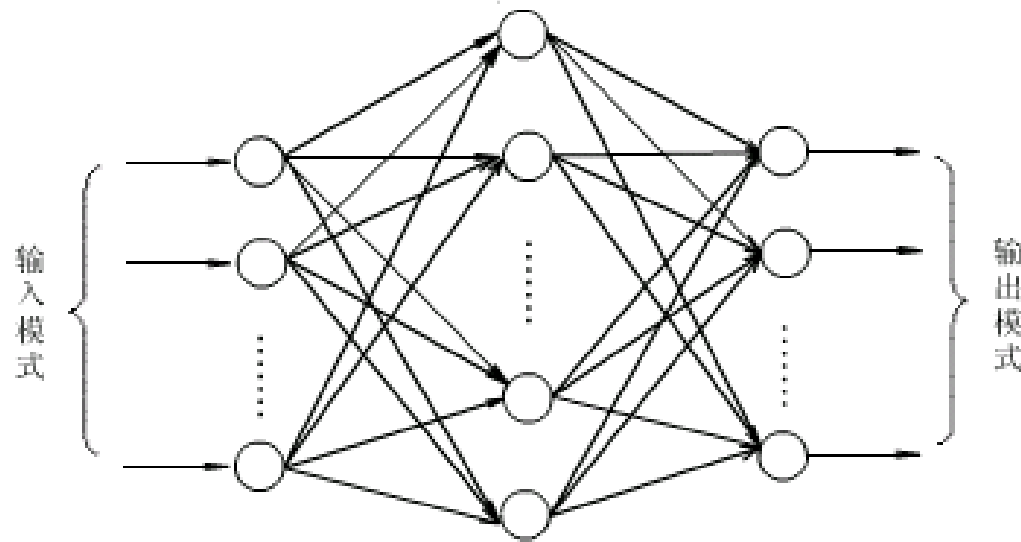
waiting Deep Learning until 2006



2.2 B-P Network

❖ Architecture

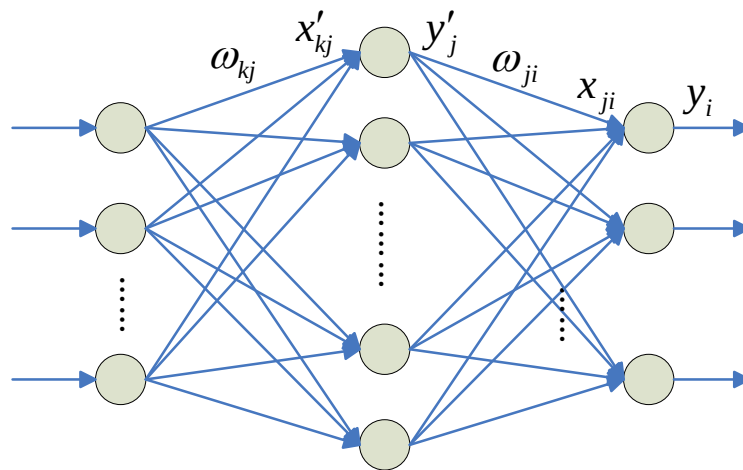
A kind of multi-layer perceptron, in which the **Sigmoid activation function** is used.



B-P Learning Algorithm

❖ Learning approach

- Input data was put forward from input layer to hidden layer, then to out layer
- **Error information** was **propagated backward** from out layer to hidden layer, then to input layer



Rumelhart & McClelland,
Nature, 1986





B-P Learning Steps

- Step1.** Select a pattern from the training set and present it to the network.
- Step2.** Compute activation of input, hidden and output neurons in that sequence.
- Step3.** Compute **the error** over the output neurons by comparing the **generated outputs** with the **desired outputs**.
- Step4.** Use the calculated error to **update** all weights in the network, such that a **global error** measure gets reduced.
- Step5.** Repeat Step1 through Step4 until the global error falls below a predefined threshold.



❖ Global Error Measure

$$e(\omega) = \frac{1}{2} \sum_{i=1}^n [d_i - y_i]^2$$

generated output

desired output

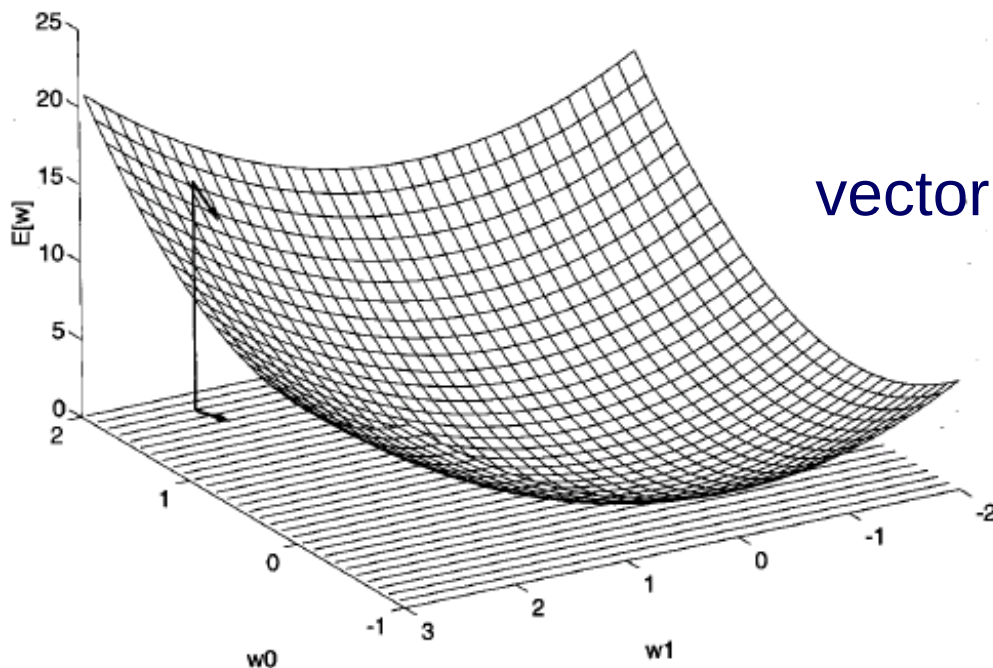
squared error

$$\omega^* = \arg \min_{\omega} e(\omega)$$

The objective is to minimize the squared error, i.e. reach the **Minimum Squared Error (MSE)**

Gradient Descent

❖ **Optimization method** for finding out the **weight vector** leading to the **MSE**



vector form:

learning rate

gradient

$$\omega = \omega + \underbrace{\eta \left[-\nabla e(\omega) \right]}_{\delta}$$

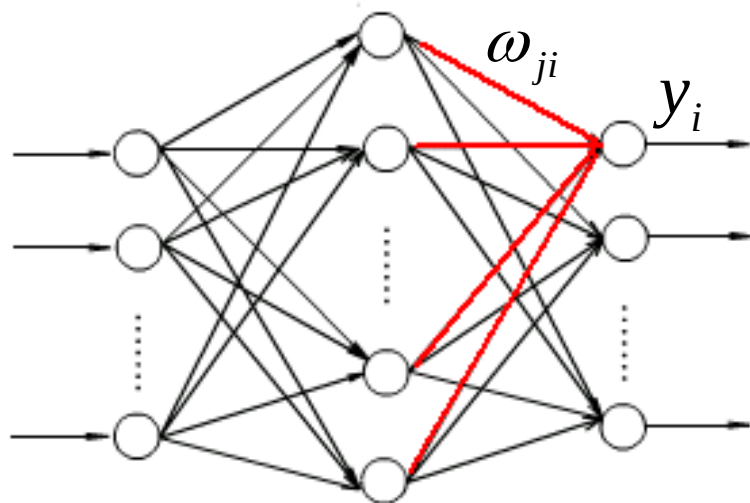
element form:

$$\omega = \omega - \eta \frac{\partial e}{\partial \omega}$$

Weight Update Rule

❖ For output layer:

$$\omega_{ji} = \omega_{ji} - \eta x_{ji} y_i (1 - y_i) (y_i - d_i)$$



$$\frac{\partial e}{\partial \omega_{ji}} = \frac{\partial e}{\partial y_i} \frac{\partial y_i}{\partial \sigma_i} \frac{\partial \sigma_i}{\partial \omega_{ji}}$$

Weight Update Rule

❖ For output layer:

$$\omega_{ji} = \omega_{ji} - \eta x_{ji} y_i (1 - y_i) (y_i - d_i)$$

$$e = \frac{1}{2} \sum (d_i - y_i)^2 \Rightarrow \frac{\partial e}{\partial y_i} = (y_i - d_i)$$

$$y_i = \frac{1}{1 + e^{-\sigma_i}} \Rightarrow \frac{\partial y_i}{\partial \sigma_i} = y_i (1 - y_i)$$

$$\sigma_i = \sum \omega_{ji} x_{ji} \Rightarrow \frac{\partial \sigma_i}{\partial \omega_{ji}} = x_{ji}$$

$$\frac{\partial e}{\partial \omega_{ji}} = x_{ji} y_i (1 - y_i) (y_i - d_i)$$

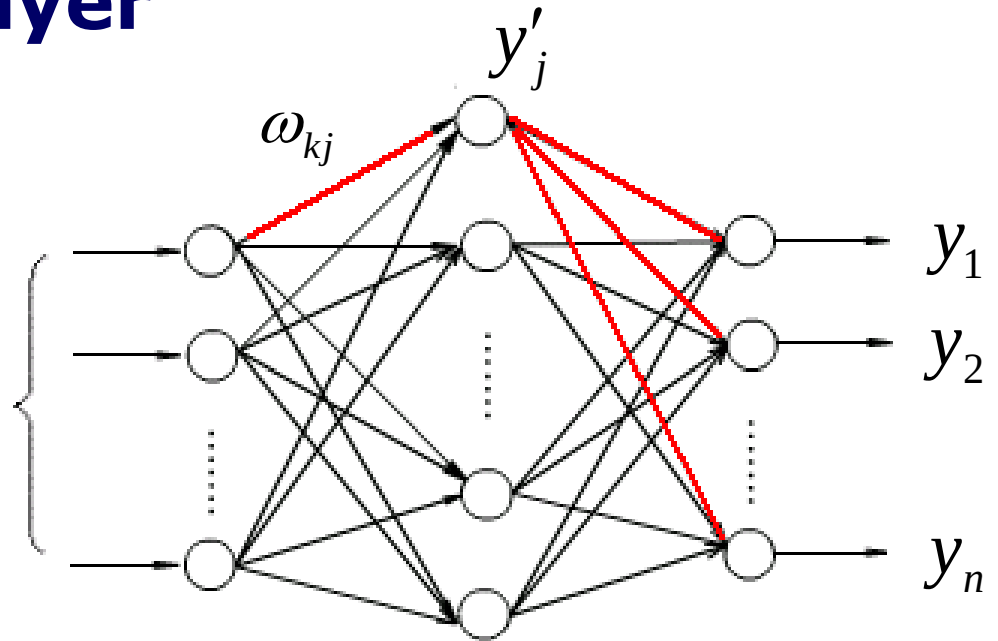
$$\Delta \omega_{ji} = -\eta \frac{\partial e}{\partial \omega_{ji}}$$



Weight Update Rule

❖ For hidden layer

$$e(\omega) = \frac{1}{2} \sum_{i=1}^n [d_i - y_i]^2$$



$$\frac{\partial e}{\partial \omega_{kj}} = \sum_{i=1}^n \frac{\partial e}{\partial y_i} \frac{\partial y_i}{\partial \sigma} \frac{\partial \sigma}{\partial y'_j} \frac{\partial y'_j}{\partial \sigma'} \frac{\partial \sigma'}{\partial \omega_{kj}}$$
$$\omega_{ji} y_i (1 - y_i) (y_i - d_i)$$

Weight Update Rule

❖ For hidden layer

$$\frac{\partial e}{\partial y_i} = (y_i - d_i), \quad \frac{\partial y_i}{\partial \sigma_i} = y_i(1 - y_i), \quad \frac{\partial \sigma_i}{\partial y'_j} = \omega_{ji}, \quad \frac{\partial y'_j}{\partial \sigma'_j} = y'_j(1 - y'_j), \quad \frac{\partial \sigma'_j}{\partial \omega_{kj}} = x_{kj}$$

$$\frac{\partial e}{\partial \omega_{kj}} = x_{kj} y'_j (1 - y'_j) \sum_{i=1}^n [\omega_{ji} y_i (1 - y_i) (y_i - d_i)] \quad \Delta \omega_{kj} = -\eta \frac{\partial e}{\partial \omega_{kj}}$$

$$\omega_{kj} = \omega_{kj} - \eta x_{kj} y'_j (1 - y'_j) \sum_{i=1}^n \omega_{ji} y_i (1 - y_i) (y_i - d_i)$$



Application1: face detection

◆ The component of vertical face recognition in RBK neural network for face detection

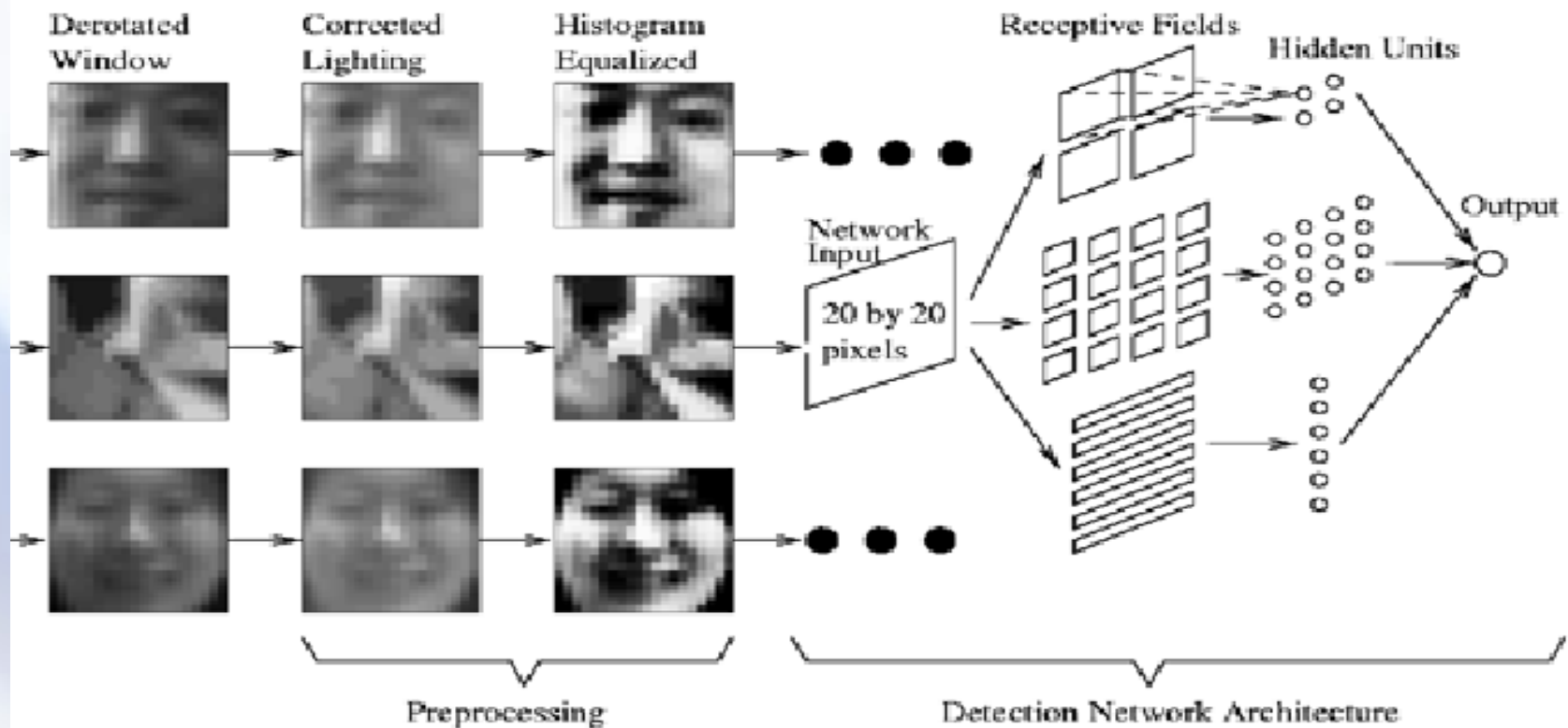
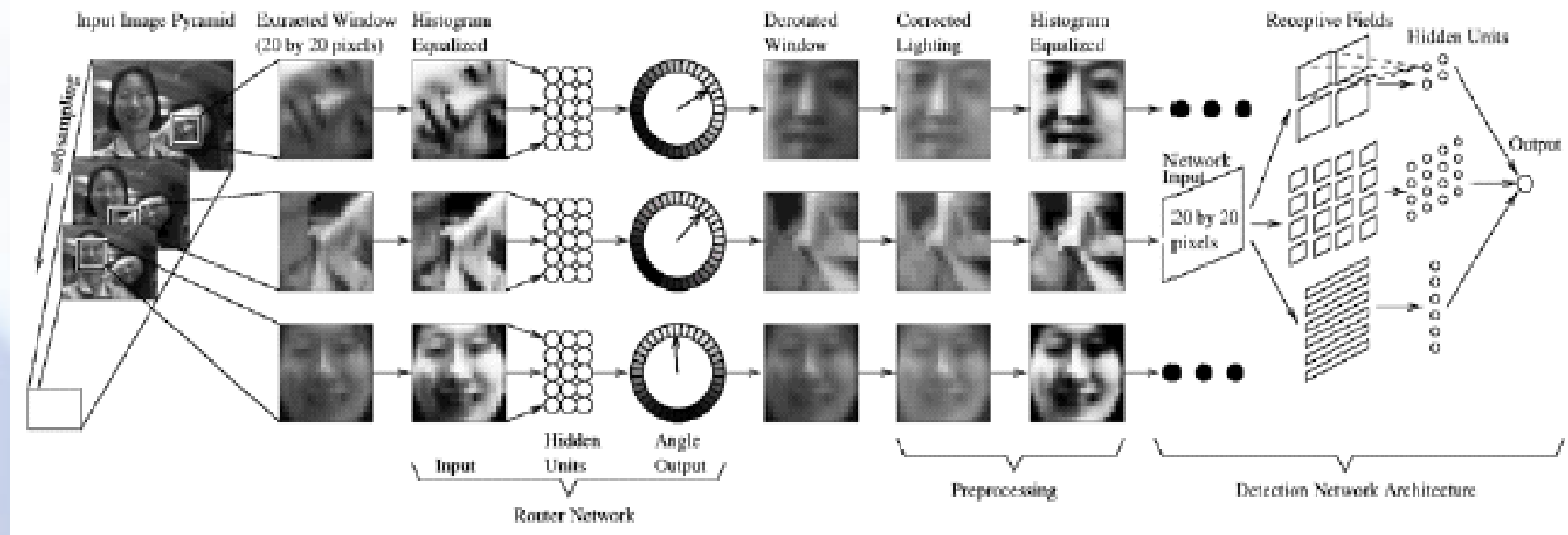


Figure from "Rotation invariant neural-network based face detection," H.A. Rowley, S. Baluja and T. Kanade, Proc. Computer Vision and Pattern Recognition, 1998, copyright 1998, IEEE

Application1: face detection

◆ The complete RBK neural network for face detection



It is composed of two neural networks. The first ANN is used to detect face directions in sub-windows. Then the sub-windows is rectified by using the detected directions to obtain vertical faces. Finally, the rectified sub-windows are put forward to the second ANN for face recognition.

Figure from “Rotation invariant neural-network based face detection,” H.A. Rowley, S. Baluja and T. Kanade, Proc. Computer Vision and Pattern Recognition, 1998, copyright 1998, IEEE

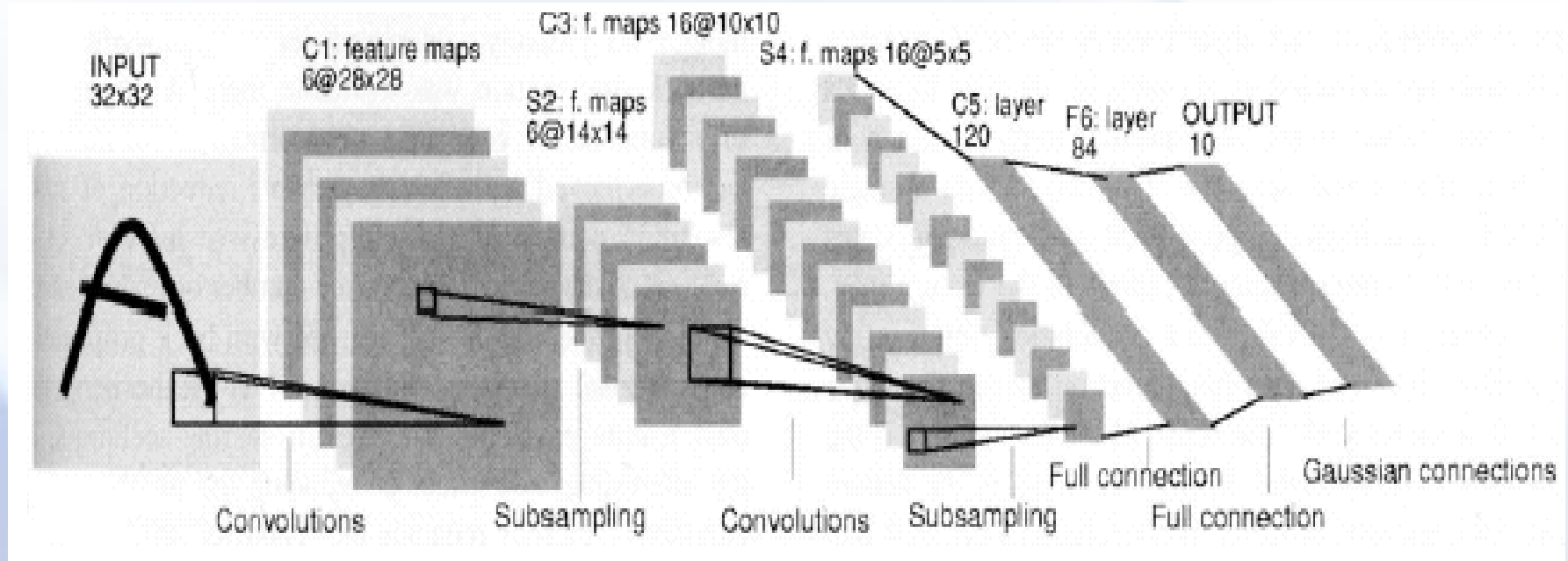
Application1: face detection

◆ The experimental results of RBK neural network for face detection

Figure from “Rotation invariant neural-network based face detection,” H.A. Rowley, S. Baluja and T. Kanade, Proc. Computer Vision and Pattern Recognition, 1998, copyright 1998, IEEE

Application2: character recognition

◆ Convolutional neural network of Lecun, called LeNet-5 for short



It includes two groups of procedures of filtering and sub-sampling, which can be thought as a feature extraction process. The recognition is performed on the output of this process.

Figure from “Gradient-Based Learning Applied to Document Recognition”, Y. Lecun et al
Proc. IEEE, 1998 copyright 1998, IEEE

Application2: character recognition

◆ LeNet-5 experimental results



Fig. 4. Size-normalized examples from the MNIST database.

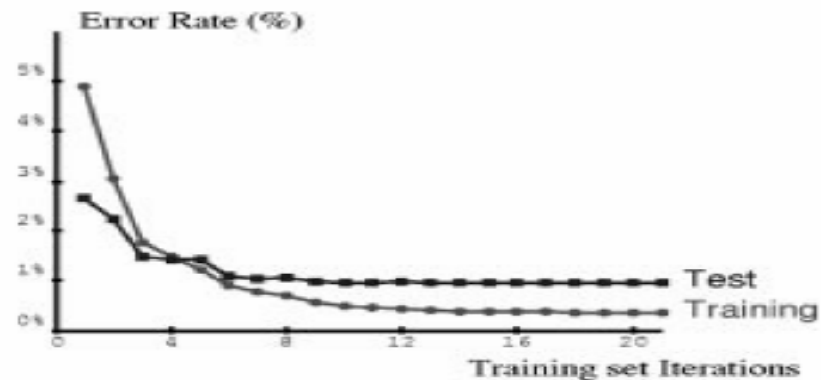


Fig. 5. Training and test error of LeNet-5 as a function of the number of passes through the 60 000 pattern training set (without distortions). The average training error is measured on-the-fly as training proceeds. This explains why the training error appears to be larger than the test error initially. Convergence is attained after 10–12 passes through the training set.

❖ 3-nearest-neighbor = 2.4% error

❖ 400×300×10 unit MLP = 1.6% error

❖ LeNet-5 = 0.9% error

❖ Current best (SVMs) ≈ 0.4% error

Figure from “Gradient-Based Learning Applied to Document Recognition”, Y. Lecun et al
Proc. IEEE, 1998 copyright 1998, IEEE



How to train deeper architectures?

- ❖ Researchers reported positive experimental results with typically two or three levels (i.e. one or two hidden layers), but training deeper networks consistently yielded poorer results.
- ❖ **Exception:** convolutional neural networks, LeNet-5





What's wrong with BP for Deep Learning?

- Early layers of MLP do not get trained well
 - Diffusion of Gradient – error attenuates as it propagates to earlier layers
 - Leads to very slow training
 - Exacerbated since top couple layers can usually learn any task "pretty well" and thus the error to earlier layers drops quickly as the top layers "mostly" solve the task– lower layers never get the opportunity to use their capacity to improve results, they just do a random feature map
 - Need a way for early layers to do effective work
- Often not enough labeled data available while there may be lots of unlabeled data
 - Can we use unsupervised/semi-supervised approaches to take advantage of the unlabeled data
- Deep networks tend to have more local minima problems than shallow networks during supervised training



Breakthrough in 2006

❖ **Deep Belief Networks (DBN)**

Hinton, G. E, Osindero, S., and Teh, Y. W. (2006). A fast learning algorithm for deep belief nets. *Neural Computation*, 18:1527-1554.

Hinton, G. E., Salakutdinov, Reducting the dimensionality of data with neural networks. *Science*, 2006, 313: 504-507.

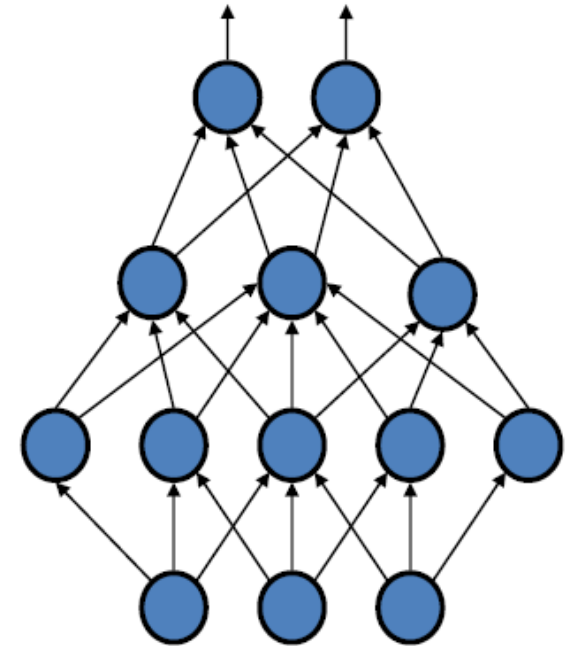
❖ **Autoencoders**

Bengio, Y., Lamblin, P., Popovici, P., Larochelle, H. (2007). Greedy Layer-Wise Training of Deep Networks, *Advances in Neural Information Processing Systems* 19



2.3 Deep Neural Network

- ❖ Simple to construct
 - Sigmoid nonlinearity for hidden layers
 - Softmax for the output layer

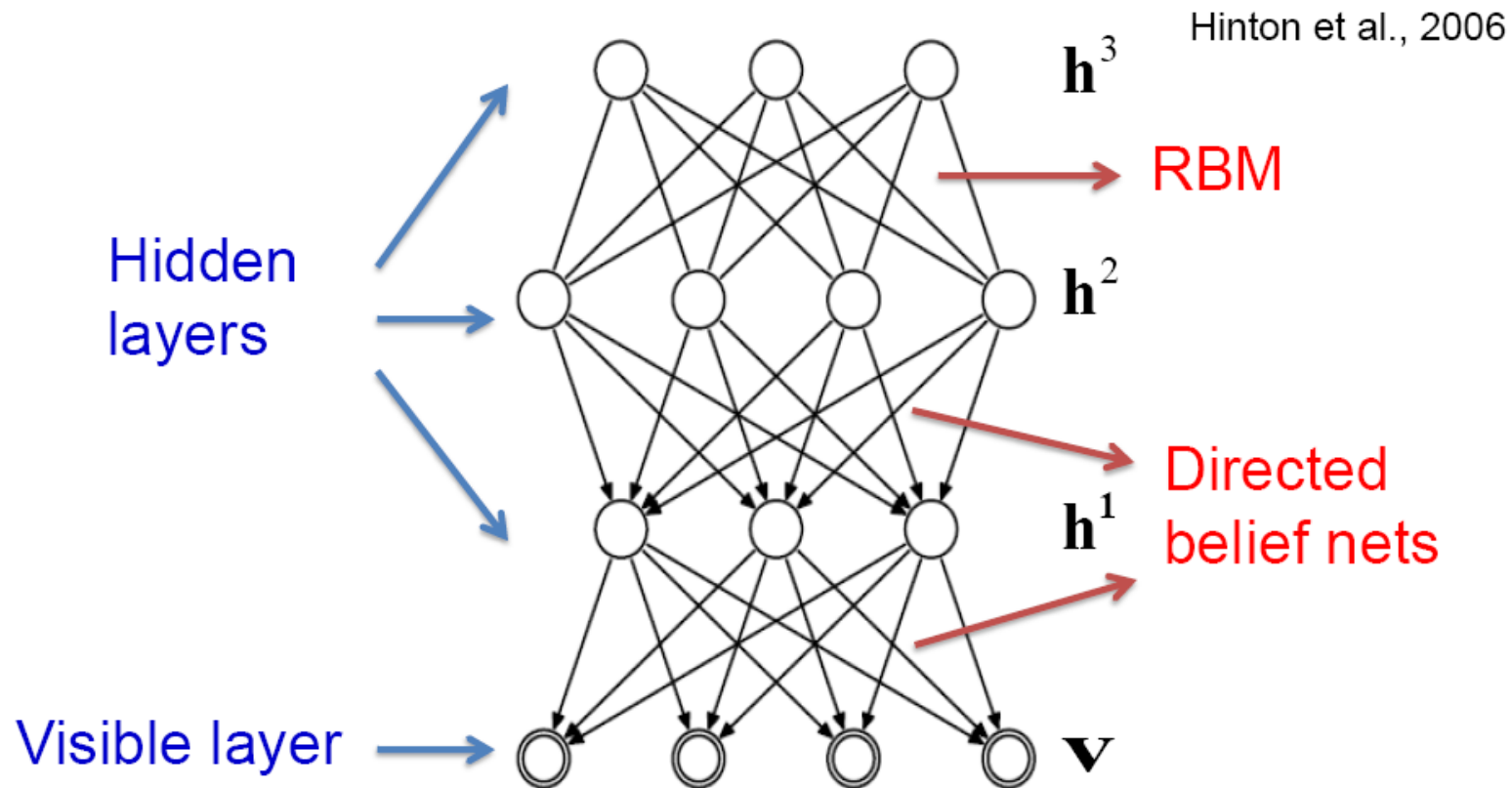


Deep Belief Network (DBN)

- ❖ Probabilistic generative models
- ❖ Contain many layers of hidden variables
- ❖ Unsupervised pre-learning provides a good initialization of the network
 - maximizing the lower-bound of the log-likelihood of the data (MLE in fact!)
- ❖ Supervised fine-tuning
 - Generative: Up-down algorithm
 - Discriminative: backpropagation



DBN Structure



$$P(\mathbf{v}, \mathbf{h}^1, \mathbf{h}^2, \dots, \mathbf{h}^l) = P(\mathbf{v} | \mathbf{h}^1) P(\mathbf{h}^1 | \mathbf{h}^2) \dots P(\mathbf{h}^{l-2} | \mathbf{h}^{l-1}) P(\mathbf{h}^{l-1}, \mathbf{h}^l)$$

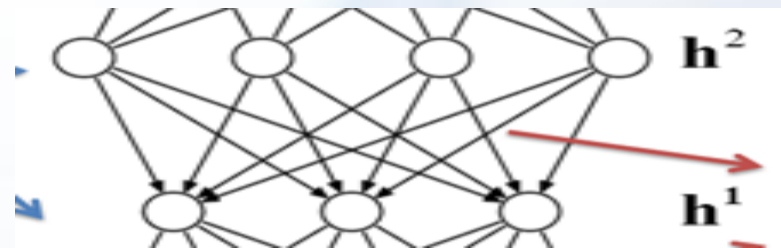
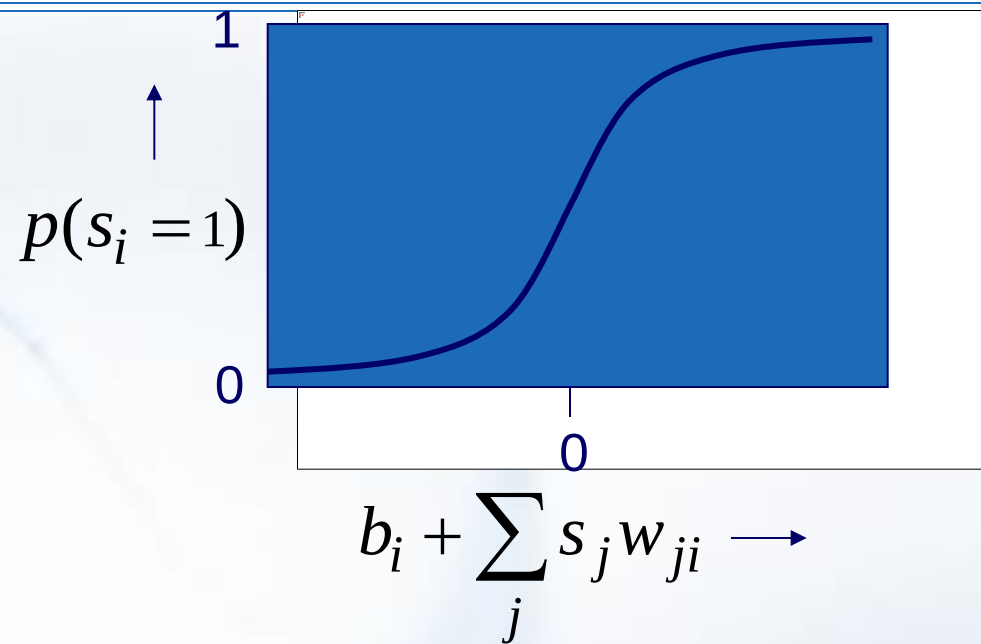
$$P(\mathbf{h}^i | \mathbf{h}^{i+1}) = \prod_{j=1}^{n^i} P(h_j^i | \mathbf{h}^{i+1})$$

Stochastic binary units (Bernoulli variables)

- ❖ These have a state of 1 or 0.
- ❖ The probability of turning on is determined by the weighted input from other units (plus a bias)

$$P(h^i | h^{i+1}) = \prod_{j=1}^{n^i} P(h_j^i | h^{i+1})$$

$$p(h_j^i = 1 | \mathbf{h}^{i+1}) = \frac{1}{1 + \exp(-b_j^i - \sum_{k=1}^{n^{i+1}} w_{kj}^i h_k^{i+1})}$$



Restricted Boltzmann Machines

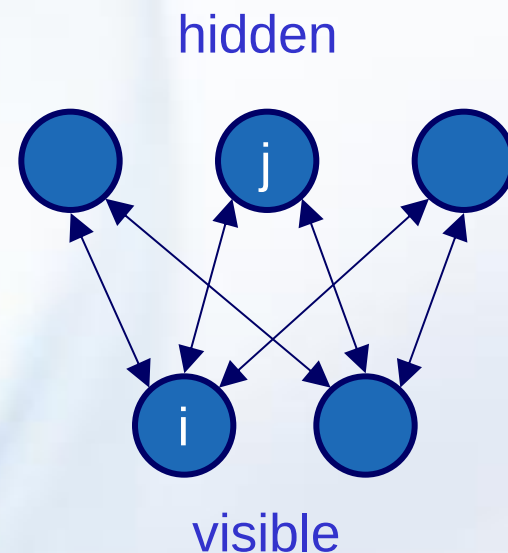
- ❖ An RBM has a single layer of hidden units which are not connected to each other and have undirected, symmetrical connections to a layer of visible units.

$$P(v, h) = \frac{1}{Z} e^{\mathbf{h}'\mathbf{W}\mathbf{v} + \mathbf{b}'\mathbf{v} + \mathbf{c}'\mathbf{h}}$$

$$\text{energy}(v, h) = -\mathbf{h}'\mathbf{W}\mathbf{v} - \mathbf{b}'\mathbf{v} - \mathbf{c}'\mathbf{h}$$

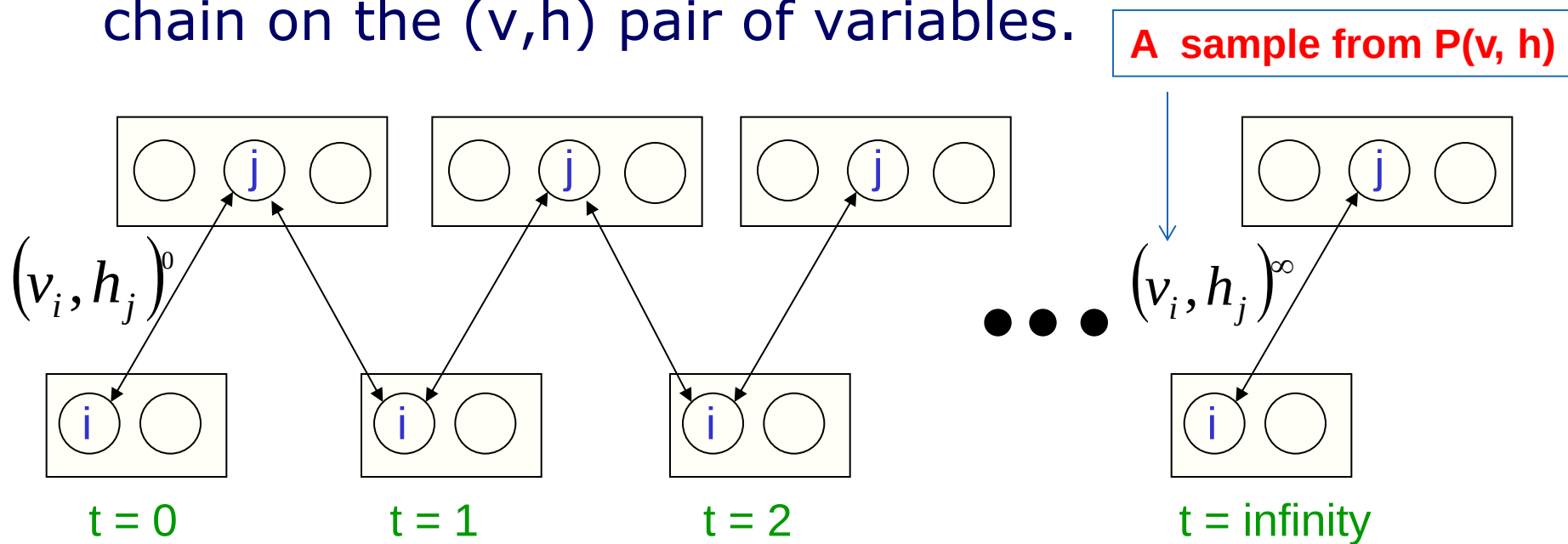
$$P(v_i = 1 | \mathbf{h}) = \text{sigm}(b_i + \sum_k W_{ki} h_k)$$

$$Q(h_j = 1 | \mathbf{v}) = \text{sigm}(c_j + \sum_k W_{jk} v_k)$$



MLE learning for RBM

- ❖ To obtain an estimator of the gradient on the log-likelihood of an RBM, we consider a Gibbs Markov chain on the (v, h) pair of variables.



Start with a training vector on the visible units.

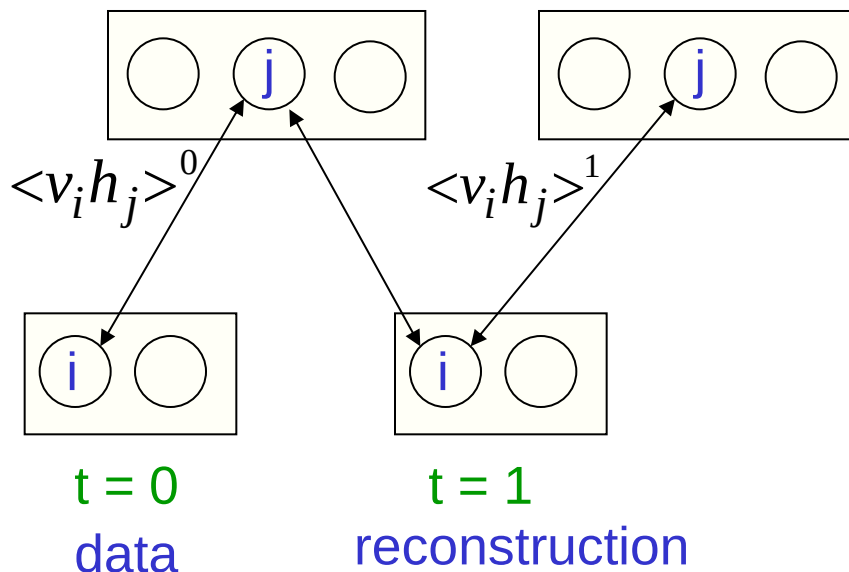
Then alternate between updating all the hidden units in parallel and updating all the visible units in parallel.

Gradient on the log-likelihood of an RBM

$$\log P(\mathbf{v}_0) = \log \sum_{\mathbf{h}} P(\mathbf{v}_0, \mathbf{h}) = \log \sum_{\mathbf{h}} e^{-\text{energy}(\mathbf{v}_0, \mathbf{h})} - \log \sum_{\mathbf{v}, \mathbf{h}} e^{-\text{energy}(\mathbf{v}, \mathbf{h})}$$
$$\frac{\partial \log P(\mathbf{v}_0)}{\partial \theta} = - \sum_{\mathbf{h}_0} Q(\mathbf{h}_0 | \mathbf{v}_0) \frac{\partial \text{energy}(\mathbf{v}_0, \mathbf{h}_0)}{\partial \theta} + \sum_{\mathbf{v}_k, \mathbf{h}_k} P(\mathbf{v}_k, \mathbf{h}_k) \frac{\partial \text{energy}(\mathbf{v}_k, \mathbf{h}_k)}{\partial \theta}$$

$k \rightarrow \infty$

❖ A quick way (**Contrastive Divergence**): to take k small (typically 1).



Start with a training vector on the visible units.

Update all the hidden units in parallel

Update the all the visible units in parallel to get a “reconstruction”.

Update the hidden units again.

RBM training algorithm

```
for all hidden units  $i$  do
    • compute  $Q(h_{0i} = 1 | \mathbf{v}_0)$  (for binomial units,  $\text{sigm}(-b_i - \sum_j W_{ij} \mathbf{v}_{0j})$ )
    • sample  $\mathbf{h}_{0i}$  from  $Q(h_{0i} = 1 | \mathbf{v}_0)$ 
end for
for all visible units  $j$  do
    • compute  $P(\mathbf{v}_{1j} = 1 | \mathbf{h}_0)$  (for binomial units,  $\text{sigm}(-c_j - \sum_i W_{ij} \mathbf{h}_{0i})$ )
    • sample  $\mathbf{v}_{1j}$  from  $P(\mathbf{v}_{1j} = 1 | \mathbf{h}_0)$ 
end for
for all hidden units  $i$  do
    • compute  $Q(h_{1i} = 1 | \mathbf{v}_1)$  (for binomial units,  $\text{sigm}(-b_i - \sum_j W_{ij} \mathbf{v}_{1j})$ )
end for
•  $W \leftarrow W - \epsilon(\mathbf{h}_0 \mathbf{v}'_0 - Q(\mathbf{h}_1 = 1 | \mathbf{v}_1) \mathbf{v}'_1)$ 
•  $b \leftarrow b - \epsilon(\mathbf{h}_0 - Q(\mathbf{h}_1 = 1 | \mathbf{v}_1))$ 
•  $c \leftarrow c - \epsilon(\mathbf{v}_0 - \mathbf{v}_1)$ 
```

Greedy Layer-wise Training of DBN

❖ Basic Idea:

- 1-level DBN is an RBM, so use RBM training
- After training the top-level RBM of a m-level DBN, insert the RBM parameters in a (m+1)-level DBN
- $Q(\mathbf{h}^m | \mathbf{h}^{m-1})$ of the RBM does not correspond to $P(\mathbf{h}^m | \mathbf{h}^{m-1})$ in the DBN, except the top layer.
- However, we can use $Q(\mathbf{h}^m | \mathbf{h}^{m-1})$ as the approximation of $P(\mathbf{h}^m | \mathbf{h}^{m-1})$
- Variational bound justifies such greedy layer-wise training of RBMs: Improvement for the next layer will yield improvement for the previous layer

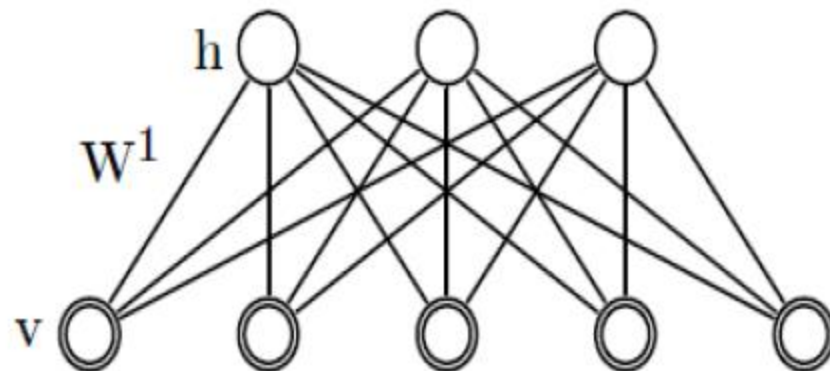
$$\log P(\mathbf{x}) \geq H_{Q(\mathbf{h}|\mathbf{x})} + \sum_{\mathbf{h}} Q(\mathbf{h}|\mathbf{x}) \underbrace{(\log P(\mathbf{h}) + \log P(\mathbf{x}|\mathbf{h}))}_{\text{Trained by the second layer RBM}}$$

Trained by the second layer RBM

Greedy Layer-wise Training of DBN

❖ First step:

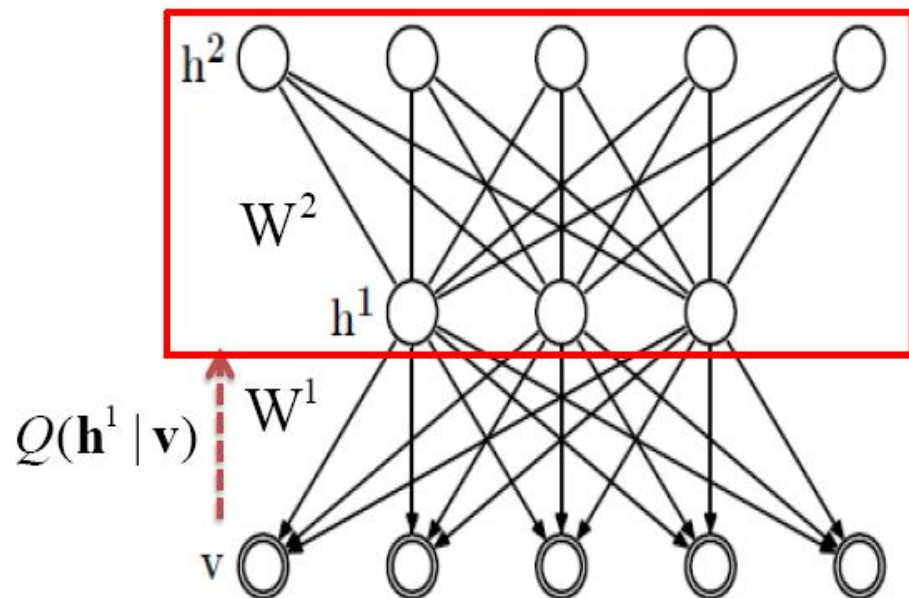
- Construct an RBM with an input layer v and a hidden layer h
- Train the RBM



Greedy Layer-wise Training of DBN

❖ Second step:

- Stack another hidden layer on top of the RBM to form a new RBM
- Fix W^1 , sample $\mathbf{h}^1$ from $Q(\mathbf{h}^1 | \mathbf{v})$ as input. Train W^2 as RBM.

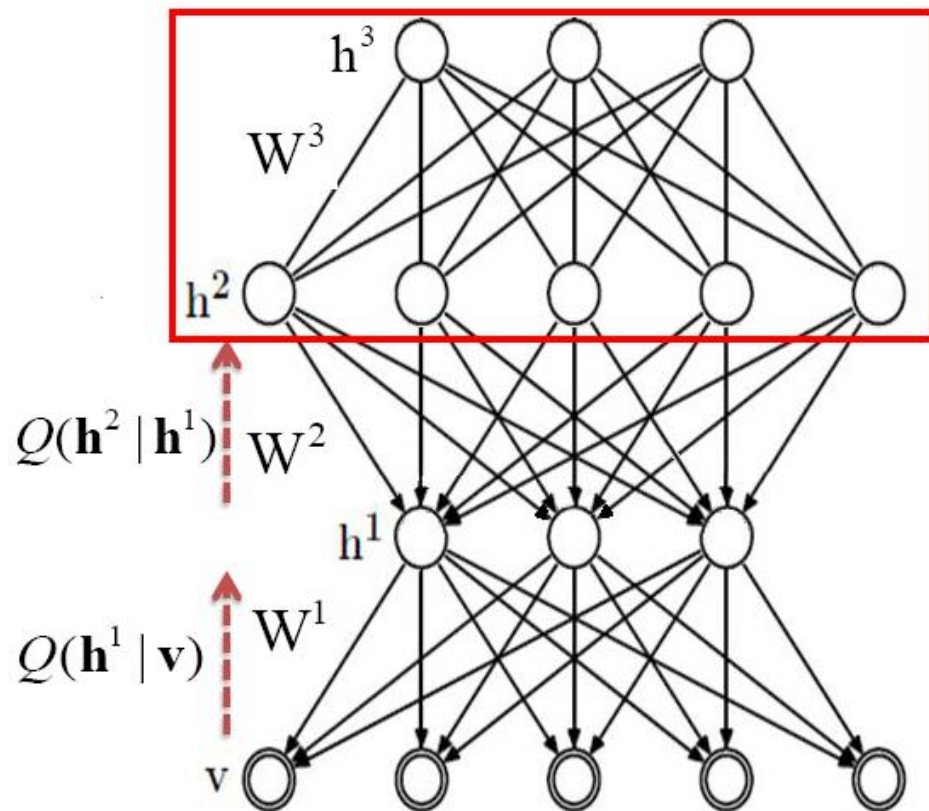


Greedy Layer-wise Training of DBN

❖ Third step:

–Continue to stack layers on top of the network, train it as previous step, with sample sampled from $Q(\mathbf{h}^2 | \mathbf{h}^1)$

❖ Continued.....



Greedy Layer-wise Training of DBN

- initialize $b^0 = 0$

for $\ell = 1$ to L do

- initialize $W^i = 0, b^i = 0$
- while not stopping criterion do
 - sample $g^0 = x$ from $\hat{p}$
 - for $i = 1$ to $\ell - 1$ do
 - sample g^i from $Q(g^i|g^{i-1})$
 - end for
 - RBMupdate($g^{\ell-1}, \epsilon, W^\ell, b^\ell, b^{\ell-1}$)
- end while

end for





Fine-tuning

- ❖ Fine-tune the parameters of all the layers together.
- ❖ Treat DBN greedy layer-wise training as “pre-training” that finds a good initial set of weights which can be fine-tuned by a **local search procedure**.
 - Contrastive wake-sleep is one way of fine-tuning the model to be **better at generation**.
- ❖ **Backpropagation** can be used to fine-tune the model for **better discrimination**.
 - This overcomes many of the limitations of standard backpropagation.





Fine-tuning with up-down algorithm

Step1. Do a stochastic bottom-up pass

- Use the MLE rule to adjust the top-down weights to be good at reconstructing the feature activities in the layer below.

Step2. Do a few iterations of sampling in the top level RBM

-- Adjust the weights in the top-level RBM.

Step3. Do a stochastic top-down pass

- Adjust the bottom-up weights to be good at reconstructing the feature activities in the layer above.



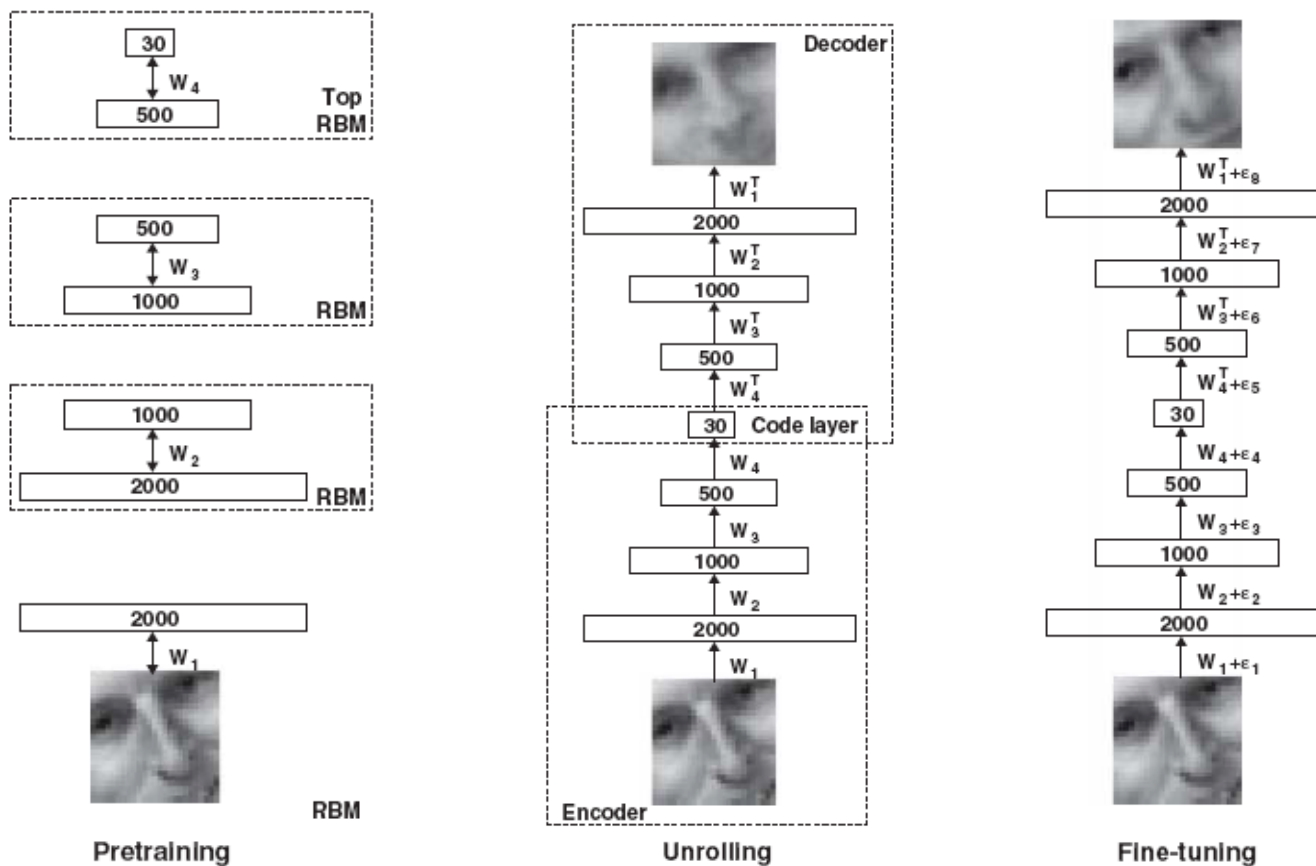
Fine-tuning with back-propagation

- ❖ Optimize all the parameters with respect to the supervised criterion C , using stochastic gradient descent.

-
- Recursively define mean-field propagation $\mu^i(x) = E[g^i | g^{i-1} = \mu^{i-1}(x)]$ where $\mu^0(x) = x$, and $E[g^i | g^{i-1} = \mu^{i-1}]$ is the expected value of g^i under the RBM conditional distribution $Q(g^i | g^{i-1})$, when the values of g^{i-1} are replaced by the mean-field values $\mu^{i-1}(x)$. In the case where g^i has binomial units, $E[g_j^i | g^{i-1} = \mu^{i-1}] = \text{sigm}(-b_j^i - \sum_k W_{jk}^i \mu_k^{i-1}(x))$.
 - Define the network output function $f(x) = V(\mu^L(x)', 1)'$
 - Iteratively minimize the expected value of $C(f(x), y)$ for pairs (x, y) sampled from $\hat{p}$ by tuning parameters W, b, V . This can be done by stochastic gradient descent with learning rate ϵ_C , using an appropriate stopping criterion such as early stopping on a validation set.
-

AutoEncoder

- ❖ The weights can be trained together by minimizing the discrepancy between the original data and its reconstruction.



Greedy AutoEncoder Training Algorithm

Goal of
AutoEncoder
Training:
accurately
reconstruct each
training data

Goal of RBM
Training: make the
distribution of the
confabulations be
the same as the
distribution of the
data

- initialize $b^0 = 0$.
- define $\mu^0(x) = x$.

for $\ell = 1$ to L do

- initialize $b^\ell = 0$.
- initialize temporary parameter vector $c^\ell = 0$.
- initialize W^ℓ by sampling from $\text{uniform}(-a, a)$, with $a = 1/n^{\ell-1}$.
- define the ℓ -th hidden layer output $\mu^\ell(x) = \text{sigm}(b^\ell + W^\ell \mu^{\ell-1}(x))$.
- define the ℓ -th hidden layer reconstruction parameter function, e.g. in the binomial case $\theta^\ell = \text{sigm}(c^\ell + W^{\ell'} \mu^\ell(x))$ is the vector of probabilities for the each bit to take value 1.

while not stopping criterion do

 for $i = 1$ to $\ell - 1$ do

 • compute $\mu^i(x)$ from $\mu^{i-1}(x)$.

 end for

- compute $\mu^\ell(x)$ from $\mu^{\ell-1}(x)$.
- compute reconstruction probability parameters θ^ℓ from $\mu^\ell(x)$.
- compute the error C in reconstructing $\mu^{\ell-1}$ from probability with parameters θ^ℓ .
- compute $\frac{\partial C}{\partial \omega}$, for $\omega = (W^\ell, b^\ell, c^\ell)$
- update layer parameters: $\omega \leftarrow \omega - \epsilon \frac{\partial C}{\partial \omega}$

 end while

end for



Example: Digit Recognition on MNIST

On the original data

- Very carefully trained backprop net with one or two hidden layers (Platt; Hinton) 1.6%
- SVM (Decoste & Schoelkopf, 2002) 1.4%
- Generative model of joint density of images and labels (+ generative fine-tuning) 1.25%
- Generative model of unlabelled digits followed by gentle backpropagation (Hinton & Salakhutdinov, Science 2006) 1.2%



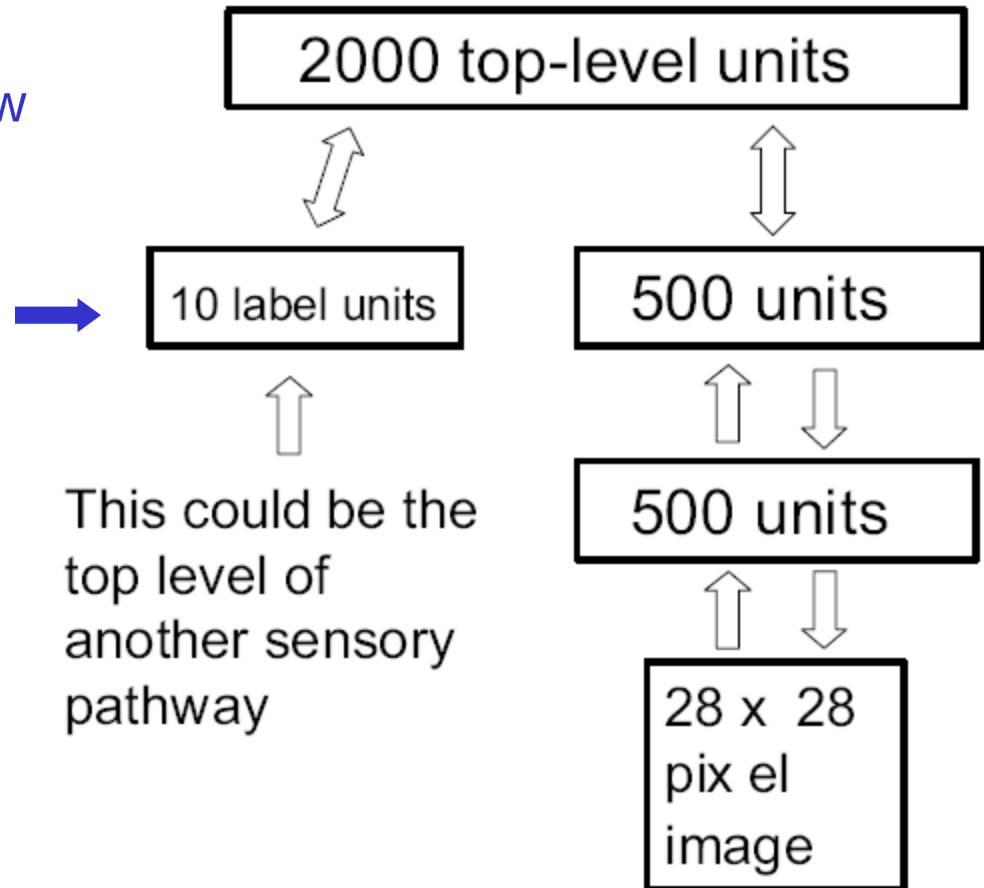
DBN for digit recognition

The top two layers form an associative memory whose energy landscape models the low dimensional manifolds of the digits.

The energy valleys have names →

The model learns to generate combinations of labels and images.

To perform recognition we start with a neutral state of the label units and do an up-pass from the image followed by a few iterations of the top-level associative memory.



Stage1: Greedy Layer-wise Training

Stage2: fine-tuning



Looking into the 'mind' of the machine

Test images and corresponding reconstruction

Test images



By
Autoencoder



By PCA





Part III: Recurrent Network



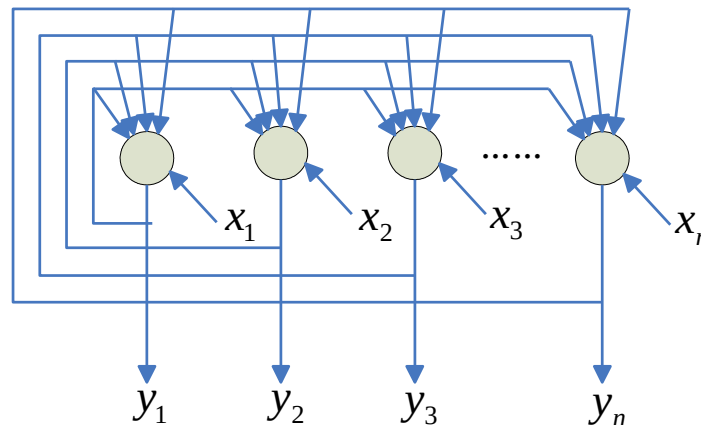
3.1 Hopfield network

- ❖ **Feedback (recurrent)** architecture

- ❖ **Dynamic System**

- ❖ **Two types**

Discrete (1982) and Continuous (*science*, 1984), by Hopfield



Attractors and Stability

- ❖ How do we “program” the solutions of the problem into **stable states (attractors)** of the network?
- ❖ How do we **ensure** that the feedback system designed is **stable**?

*Lyapunov’s modern stability theory allows us to investigate the stability problem by making use of a continuous scalar function of the state vector, called a **Lyapunov (Energy) Function**.*



Energy Function for Hopfield Network

❖ With input

$$E = -\frac{1}{2} \sum_{i=0}^n \sum_{j=0}^n \omega_{ij} s_i s_j - \sum_{i=1}^n I_i s_i$$

❖ Without input

$$E = -\frac{1}{2} \sum_{i=0}^n \sum_{j=0}^n \omega_{ij} s_i s_j = -\frac{1}{2} \mathbf{S}^T \boldsymbol{\omega} \mathbf{S}$$



Hopfield model

❖ It is proven by Hopfield that asynchronous Hopfield network with

$$w_{ij} = \begin{cases} w_{ji}, i \neq j \\ 0, i = j \end{cases}$$

is stable. How?

*Whatever be the initial state of the network, the **energy** decreases continuously with time until the system settles down into any **local minimum** of the energy surface.*



Hopfield Network Application: *Associative Memories*

- ❖ A primary application of Hopfield network
- ❖ **Recalls data (attractor state)** based on an input that **partially resembles the data itself.**
- ❖ Also named **content-addressable memory.**



Stored Pattern



Memory Association

虞台文,
*Feedback
Networks
and
Associative
Memories*

Hopfield Network: *Associative Memories*

- ❖ A primary application of Hopfield network
- ❖ **Recalls data (attractor state)** based on an input that **partially resembles the data itself.**
- ❖ Also named **content-addressable memory.**



Stored Pattern

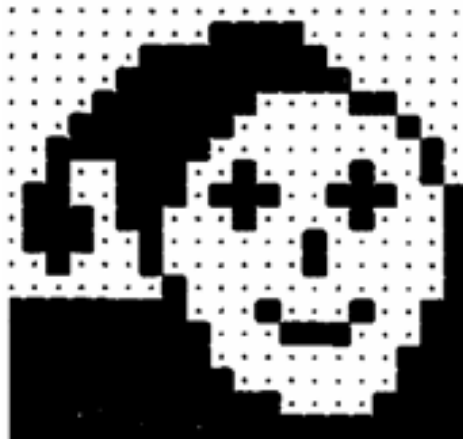


Memory Association

From 虞台文,
*Feedback
Networks
and
Associative
Memories*

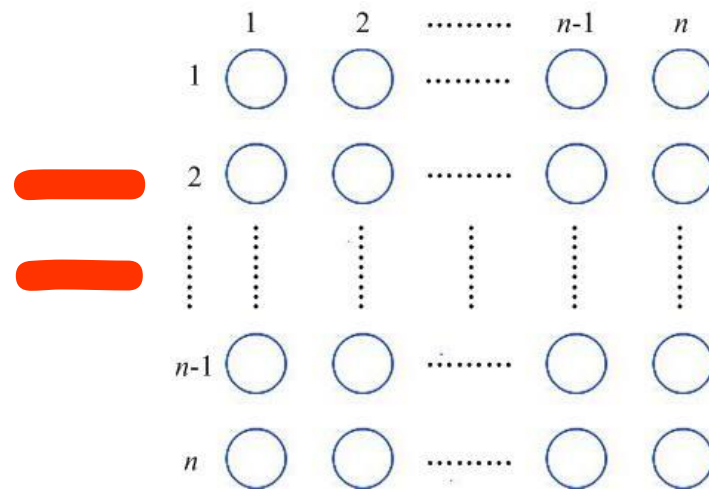
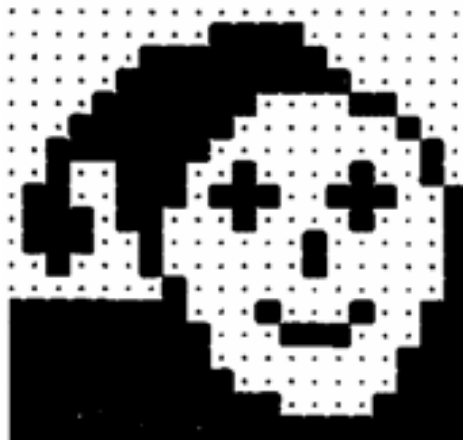


How to store patterns?



= ?

How to store patterns?



n^2 : Dimension of the stored pattern

Weight determination: Outer Product

❖ **Vector form:**

$$\omega = \sum_{k=1}^K \mathbf{X}_k \mathbf{X}_k^T - K\mathbf{I}$$

❖ **Element form:**

$$\omega_{ij} = \begin{cases} \sum_{k=1}^K x_{ik} x_{jk}, & i \neq j \\ 0, & i = j \end{cases}$$

❖ **Why?**

Satisfy the Hopfield model



An example of Hopfield memory

From 虞台文,
*Feedback
Networks
and
Associative
Memories*

$$\omega = \sum_{k=1}^K \mathbf{X}_k \mathbf{X}_k^T - KI$$

$$\mathbf{x}^1 = (1, -1, -1, 1)^T$$


$$\mathbf{x}^2 = (-1, 1, -1, 1)^T$$


$$\mathbf{W} = \begin{bmatrix} 0 & -2 & 0 & 0 \\ -2 & 0 & 0 & 0 \\ 0 & 0 & 0 & -2 \\ 0 & 0 & -2 & 0 \end{bmatrix}$$

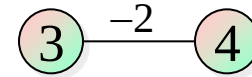
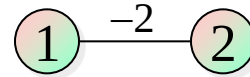
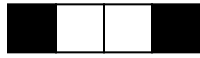
$$\mathbf{x}^1(\mathbf{x}^1)^T = \begin{bmatrix} 1 & -1 & -1 & 1 \\ -1 & 1 & 1 & -1 \\ -1 & 1 & 1 & -1 \\ 1 & -1 & -1 & 1 \end{bmatrix}$$

$$\mathbf{x}^2(\mathbf{x}^2)^T = \begin{bmatrix} 1 & -1 & 1 & -1 \\ -1 & 1 & -1 & 1 \\ 1 & -1 & 1 & -1 \\ -1 & 1 & -1 & 1 \end{bmatrix}$$

$$\mathbf{x}^1(\mathbf{x}^1)^T + \mathbf{x}^2(\mathbf{x}^2)^T = \begin{bmatrix} 2 & -2 & 0 & 0 \\ -2 & 2 & 0 & 0 \\ 0 & 0 & 2 & -2 \\ 0 & 0 & -2 & 2 \end{bmatrix}$$

The example contd.

$$\mathbf{x}^1 = (1, -1, -1, 1)^T$$



$$\mathbf{x}^2 = (-1, 1, -1, 1)^T$$

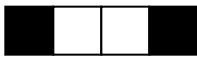


$$\mathbf{W} = \begin{bmatrix} 0 & -2 & 0 & 0 \\ -2 & 0 & 0 & 0 \\ 0 & 0 & 0 & -2 \\ 0 & 0 & -2 & 0 \end{bmatrix}$$

$$\begin{aligned} E(\mathbf{x}) &= -\frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n w_{ij} x_i x_j - \sum_{i=1}^n I_i x_i \\ &= -\frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n w_{ij} x_i x_j \\ &= -\frac{1}{2} \mathbf{x}^T \mathbf{W} \mathbf{x} \\ &= 2(x_1 x_2 + x_3 x_4) \end{aligned}$$

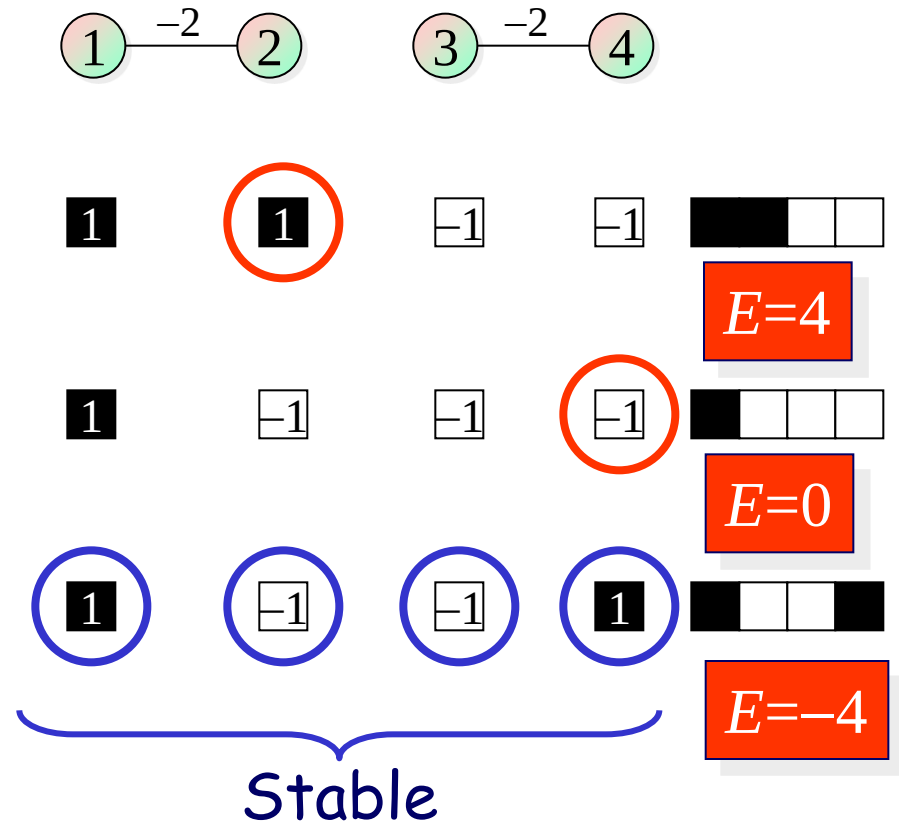


Recall the first pattern ($\mathbf{x}^1$)

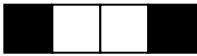
$$\mathbf{x}^1 = (1, -1, -1, 1)^T$$


$$\mathbf{x}^2 = (-1, 1, -1, 1)^T$$


$$E(\mathbf{x}) = 2(x_1x_2 + x_3x_4)$$



Recall the second pattern ($\mathbf{x}^2$)

$$\mathbf{x}^1 = (1, -1, -1, 1)^T$$


$$1 \xrightarrow{-2} 2$$

$$3 \xrightarrow{-2} 4$$

$$\mathbf{x}^2 = (-1, 1, -1, 1)^T$$


$$1$$

$$1$$

$$-1$$

$$-1$$



$$E=4$$

$$-1$$

$$1$$

$$-1$$

$$-1$$



$$E=0$$

$$E(\mathbf{x}) = 2(x_1x_2 + x_3x_4)$$

$$-1$$

$$1$$

$$-1$$

$$1$$



$$E=-4$$

Stable



Hopfield Network Application: *Combinatorial Optimization*

- ❖ **Another primary application of Hopfield network**
- ❖ **Transform the objective function into the **energy function**. Then the attractor state of the network is the solution of the optimization problem.**



An example: Solve Traveling Salesman Problem (TSP)

Given n cities with distances d_{ij} , what is the **shortest** tour?

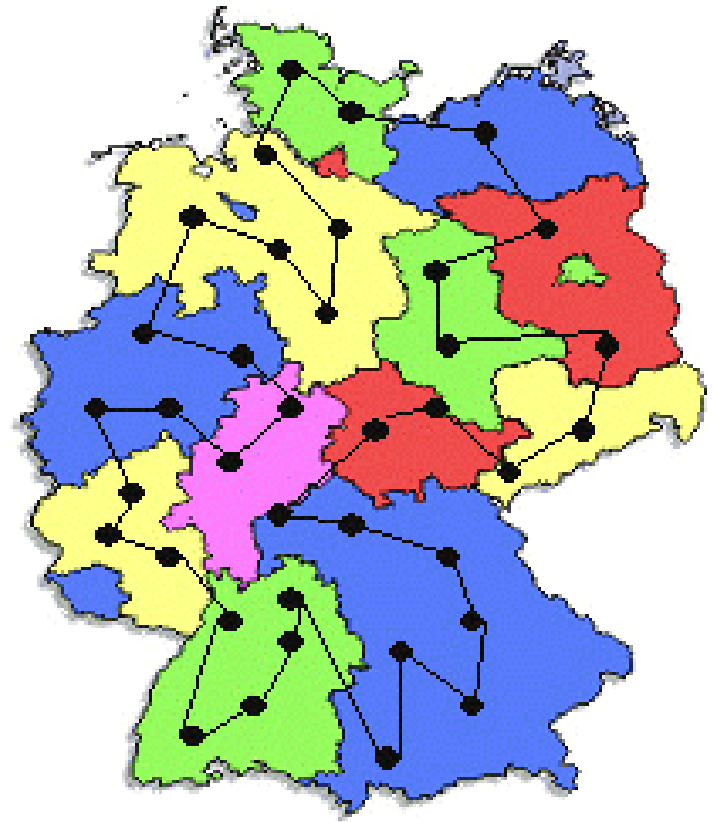
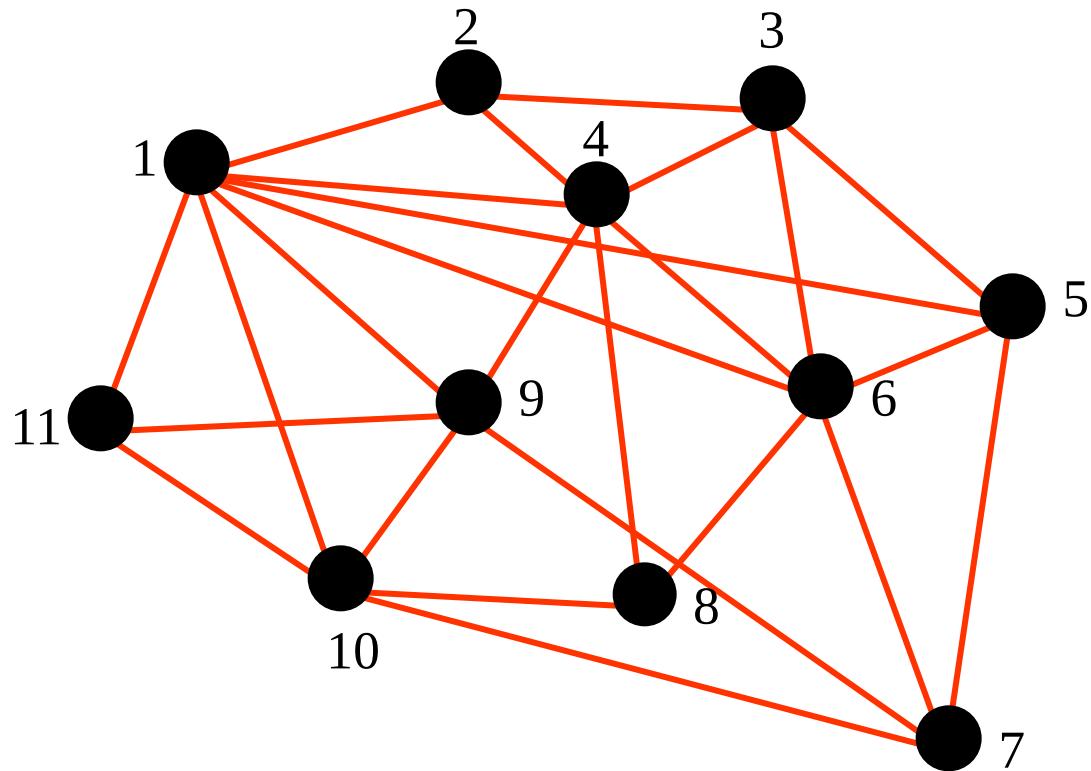
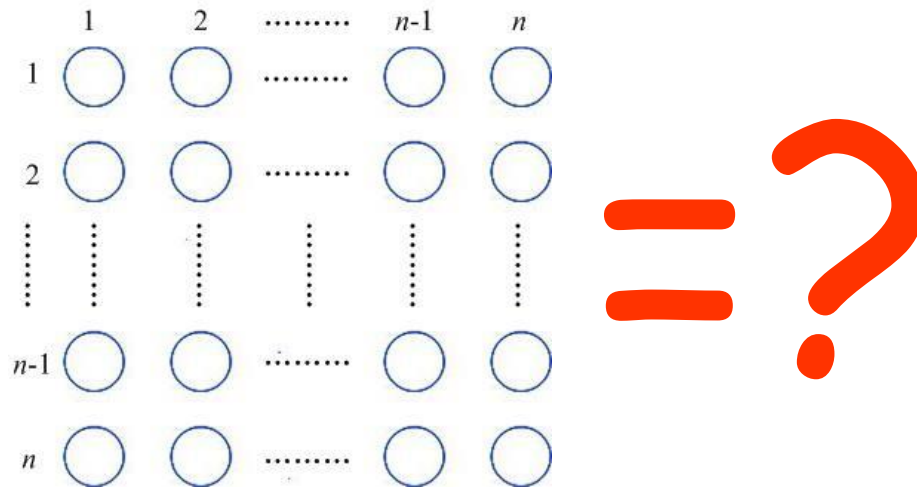


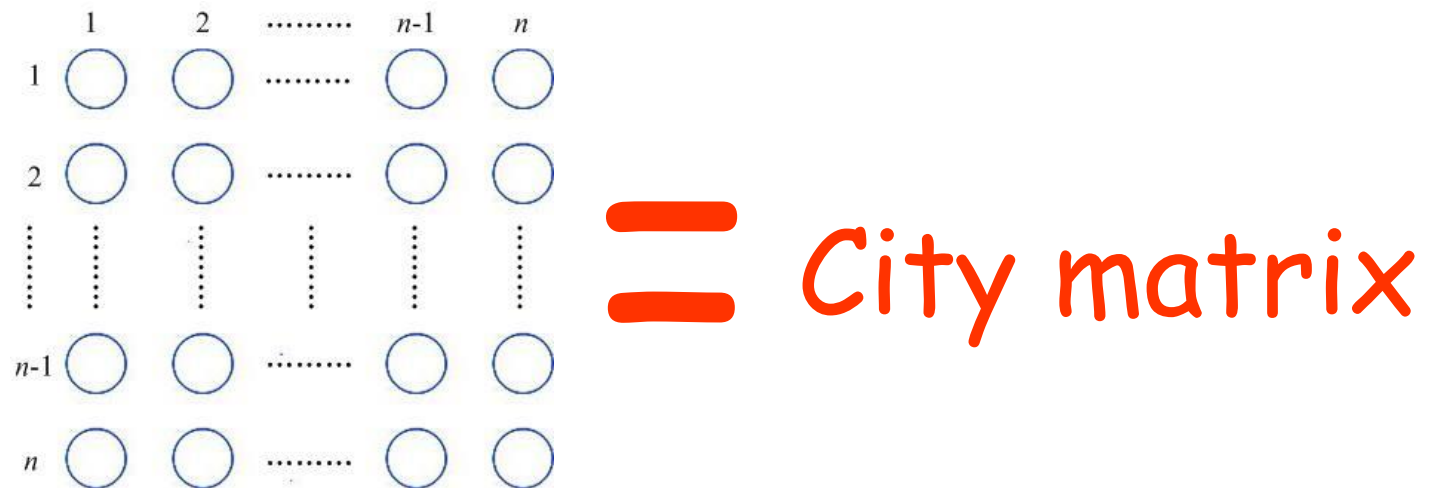
Illustration of TSP Graph



Hopfield Network for TSP



Hopfield Network for TSP

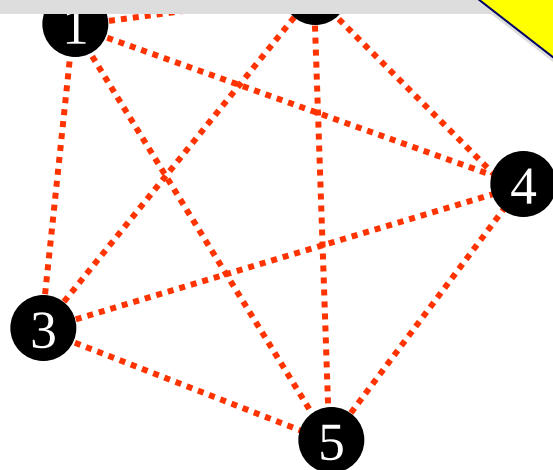


- Constraint

1. Each row can have only one neuron "on".
2. Each column can have only one neuron "on".
3. For a n -city problem, n neurons will be on.

Hopfield Network for TSP

The salesman reaches
city 5 at time 3.



City

Time

	1	2	3	4	5
1	☐	☐	☐	☐	☐
2	☐	☐	☐	☐	☐
3	☐	☐	☐	☐	☐
4	☐	☐	☐	☐	☐
5	☐	☐	☒	☐	☐



Weight determination for TSP: *Design Energy Function*

$$E = \underbrace{\frac{\lambda_1}{2} \sum_{i=1}^n \sum_{x=1}^n \sum_{y=1}^n s_{xi} d_{xy} (s_{y,i-1} + s_{y,i+1})}_{\text{goal}}$$

$$+ \underbrace{\frac{\lambda_2}{2} \sum_{x=1}^n \sum_{i=1}^n \sum_{j=1}^n s_{xi} s_{xj} + \frac{\lambda_3}{2} \sum_{i=1}^n \sum_{x=1}^n \sum_{y=1}^n s_{xi} s_{yi} + \frac{\lambda_4}{2} \left(\sum_{x=1}^n \sum_{i=1}^n s_{xi} - n \right)^2}_{\text{constraint}}$$

Constraint-1

Each row can have only one neuron “on”.

Constraint-2

Each column can have only one neuron “on”.

Constraint-3

For a n -city problem, n neurons will be on.



Transform Energy Function into the form of 2D Hopfield Network

$$E = -\frac{1}{2} \sum_{i=0}^n \sum_{j=0}^n \omega_{ij} s_i s_j - \sum_{i=1}^n I_i s_i$$

$$E = -\frac{1}{2} \sum_{x=1}^n \sum_{i=1}^n \sum_{y=1}^n \sum_{j=1}^n s_x \omega_{xi,yj} s_{yj} - \frac{1}{2} \sum_{x=1}^n \sum_{i=1}^n I_{xi} s_{xi}$$

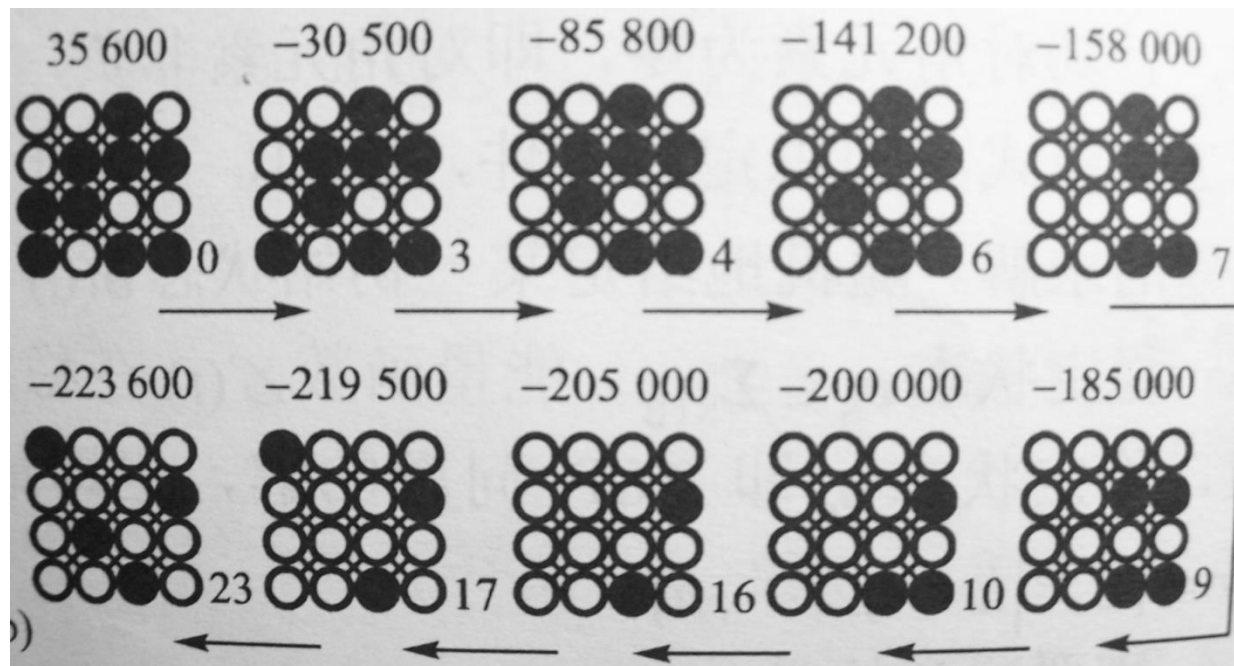
$$\omega_{xi,yj} = -\lambda_1 d_{xy} (\delta_{i,j+1} + \delta_{i,j-1}) - \lambda_2 \delta_{xy} (1 - \delta_{ij}) - \lambda_3 \delta_{ij} (1 - \delta_{xy}) - \lambda_4 (1 - \delta_{ij}) (1 - \delta_{xy})$$

$$I_{xi} = \lambda_4 n \quad \delta_{xy} = \begin{cases} 1, x = y \\ 0, x \neq y \end{cases}$$

Network is
built!

Iteration of Hopfield network for TSP problem

- The *initial state* generated randomly goes to the *stable state* (solution) with *minimum energy*

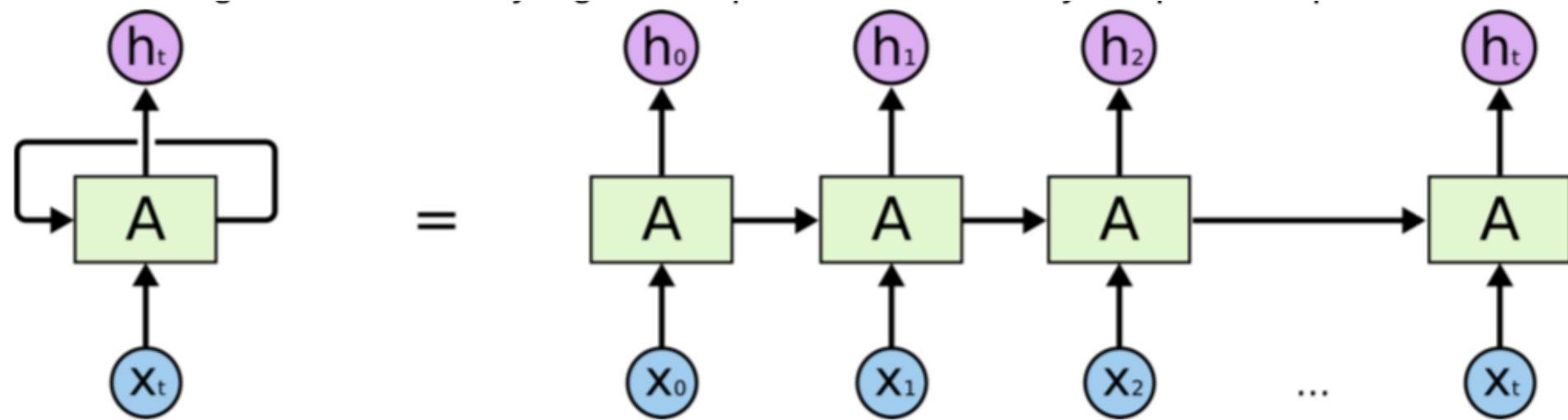


A 4-city
example

From 阮晓刚,
神经计算科学,
2006

3.2 Long Short-Term Memory

We can **unroll a RNN in time**, in this way it can be considered as a deep neural network with indefinitely many layers and can be trained by performing Back-Propagation Through Time (BPTT).

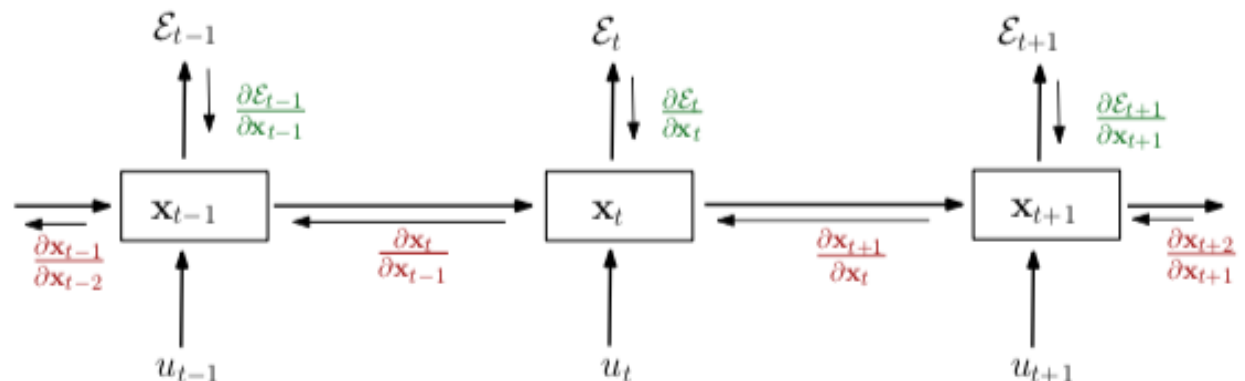
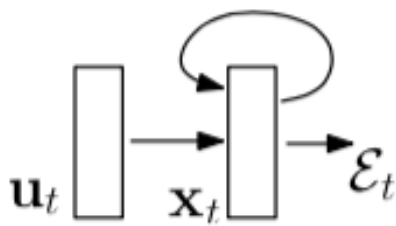


Back-Propagation Through Time (BPTT)

$$\frac{\partial \mathcal{E}}{\partial \theta} = \sum_{1 \leq t \leq T} \frac{\partial \mathcal{E}_t}{\partial \theta}$$

$$\frac{\partial \mathcal{E}_t}{\partial \theta} = \sum_{1 \leq k \leq t} \frac{\partial \mathcal{E}_t}{\partial x_t} \frac{\partial x_t}{\partial x_k} \frac{\partial x_k}{\partial \theta}$$

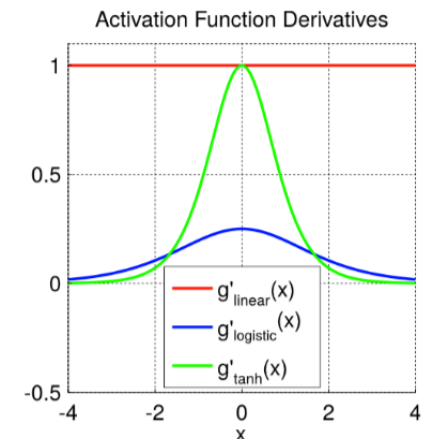
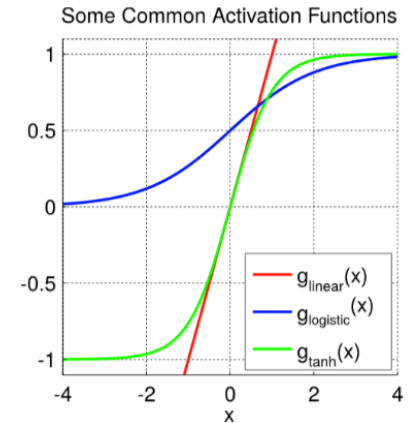
$$\frac{\partial x_t}{\partial x_k} = \prod_{t \geq i > k} \frac{\partial x_i}{\partial x_{i-1}} = \prod_{t \geq i > k} w_{rec}^T \text{diag}(\sigma'(x_{i-1}))$$



Vanishing/Exploding Gradients Problem

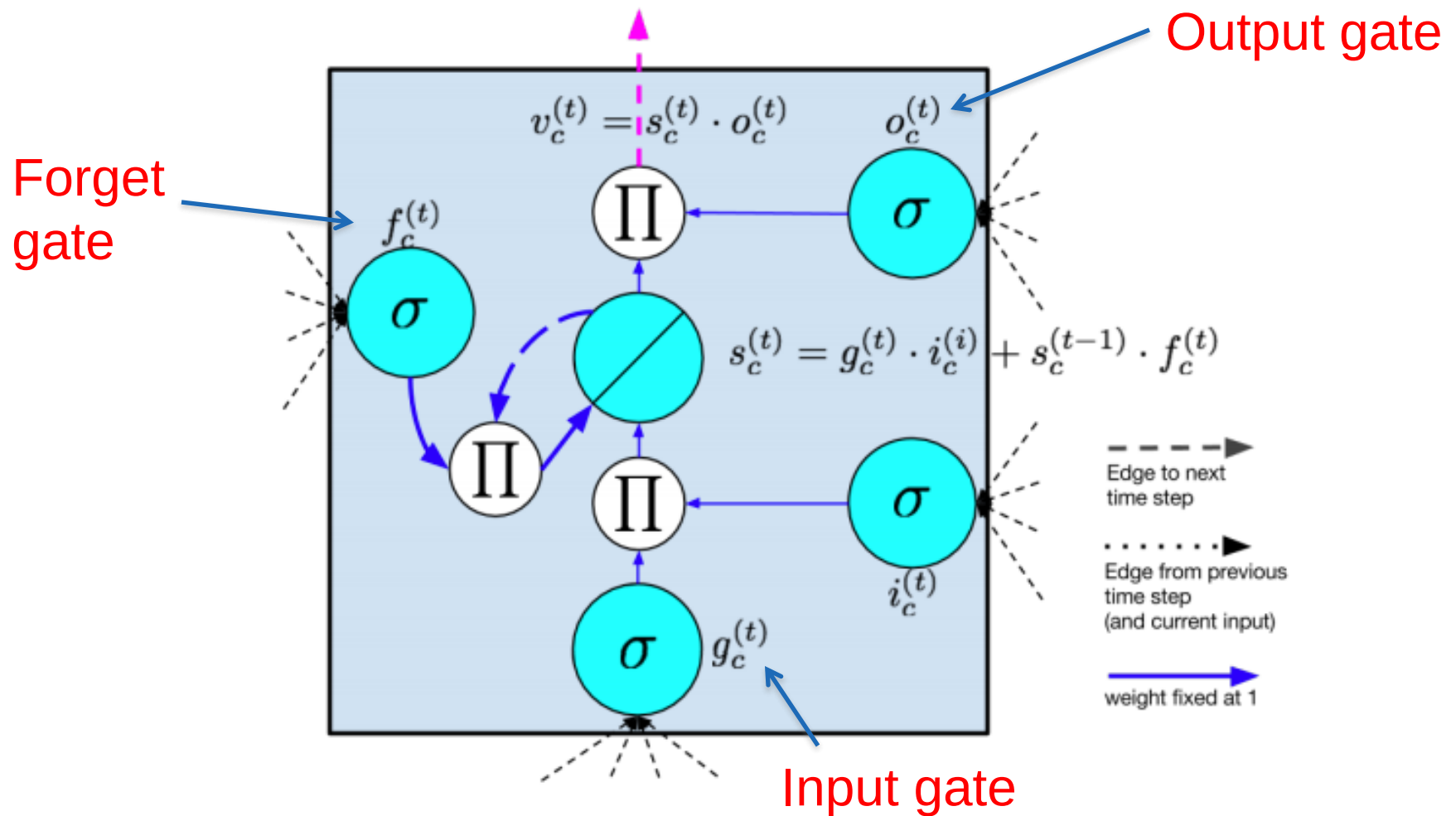
$$\frac{\partial \varepsilon_t}{\partial \theta} = \sum_{1 \leq k \leq t} \frac{\partial \varepsilon_t}{\partial x_k} \left(\prod_{t \geq i > k} w_{rec}^T \text{diag}(\sigma'(x_{i-1})) \right) \frac{\partial x_k}{\partial \theta}$$

- tanh activation function maps all values into a range between -1 and 1, and the derivative is bounded by 1.
- With multiple matrix multiplications, **gradient values shrink exponentially**, gradient contributions from “far away” steps become zero.
- Depending on activation functions and network parameters, **gradients could explode** instead of vanishing.



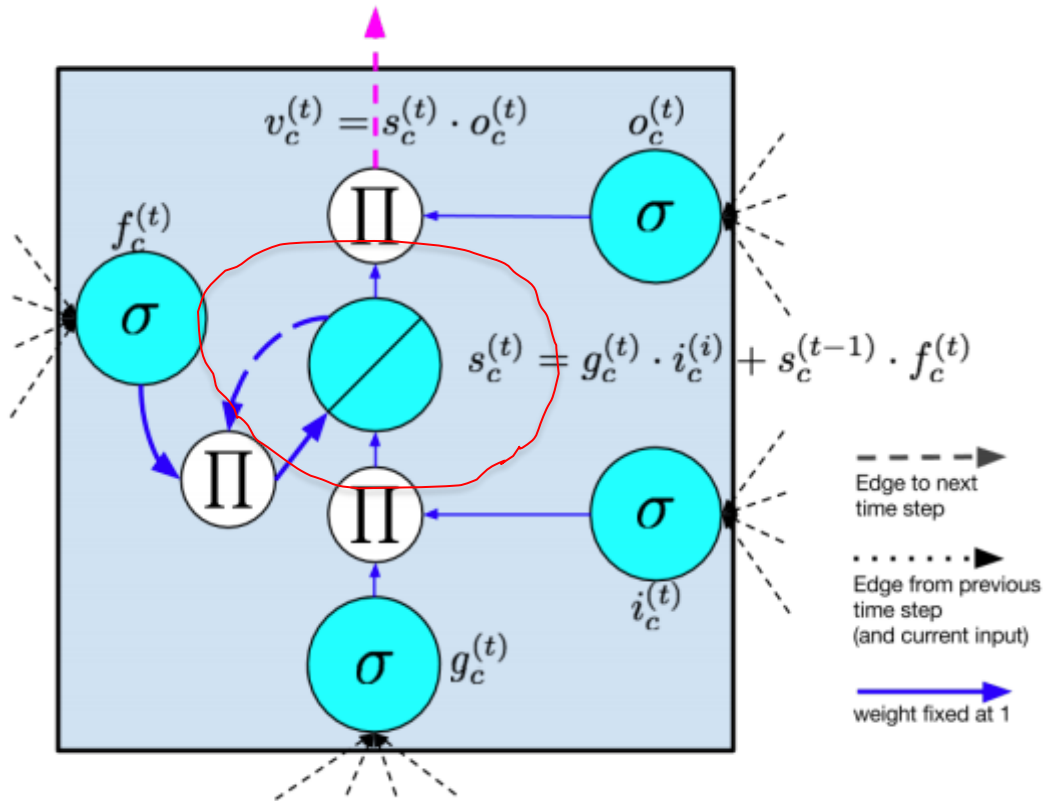
Solution: LSTM

LSTM Memory Cell



Differentiable and continuous analogy to digital memory blocks

Constant Error carousel



$$e_j(t) = f'_j(net_j(t))e_j(t+1)\omega_{jj}$$

$$f'_j(net_j(t))\omega_{jj} = 1.0$$

$$f_j(net_j(t)) = net_j(t) / \omega_{jj}$$

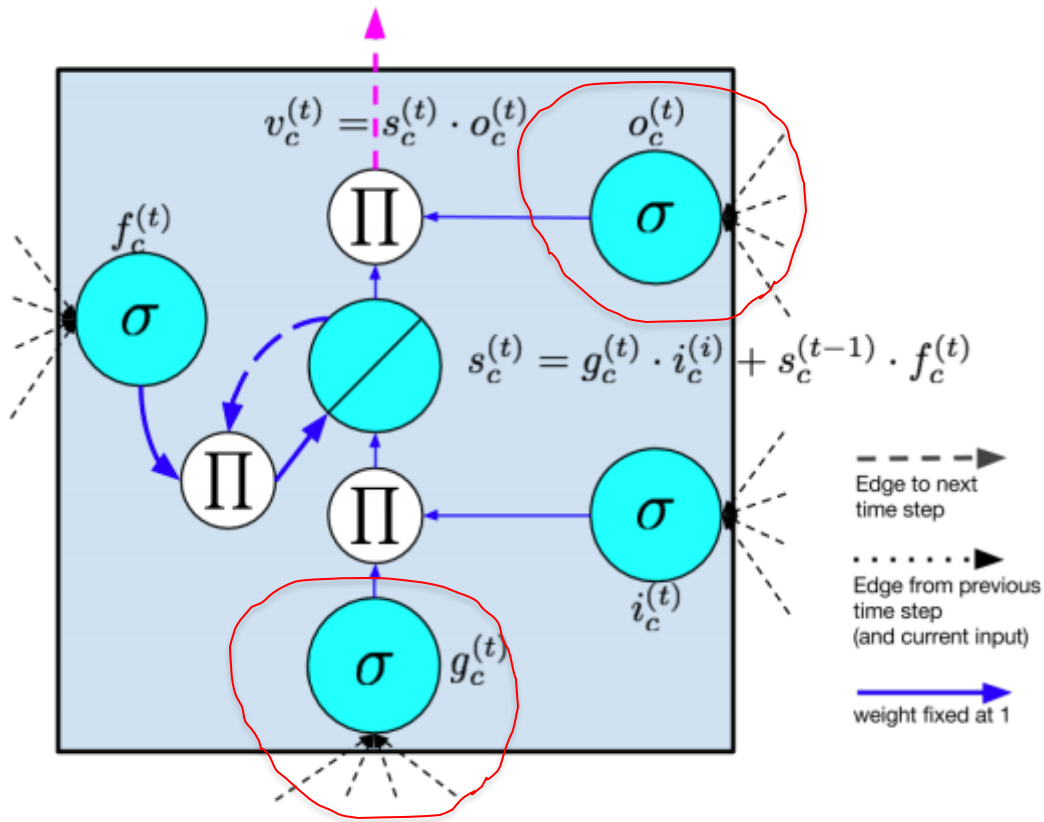
$$\begin{aligned} y_j(t+1) &= f_j(net_j(t+1)) \\ &= f_j(\omega_{jj}y_j(t)) = y_j(t) \end{aligned}$$

$$f_j(x) = x \quad \omega_{jj} = 1$$

Allow the network to propagate errors without modification



Input and output gate



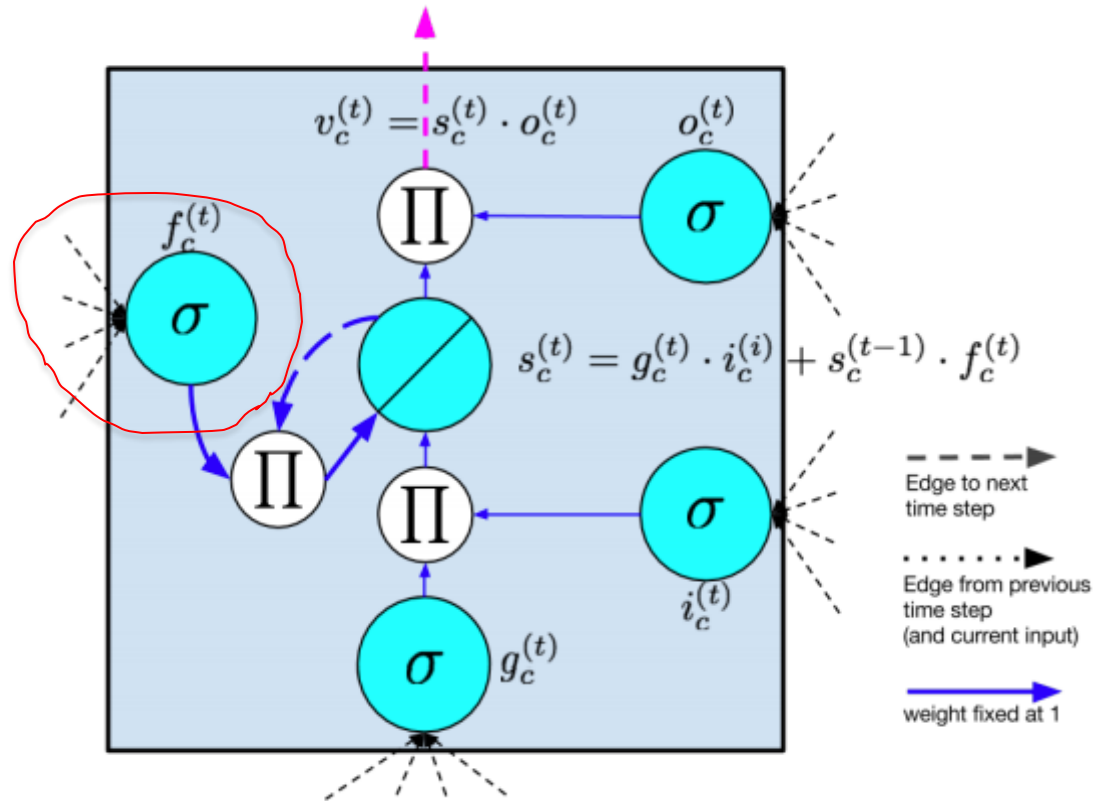
Output gate:

- Access information in memory
- Block irrelevant information

Input gate:

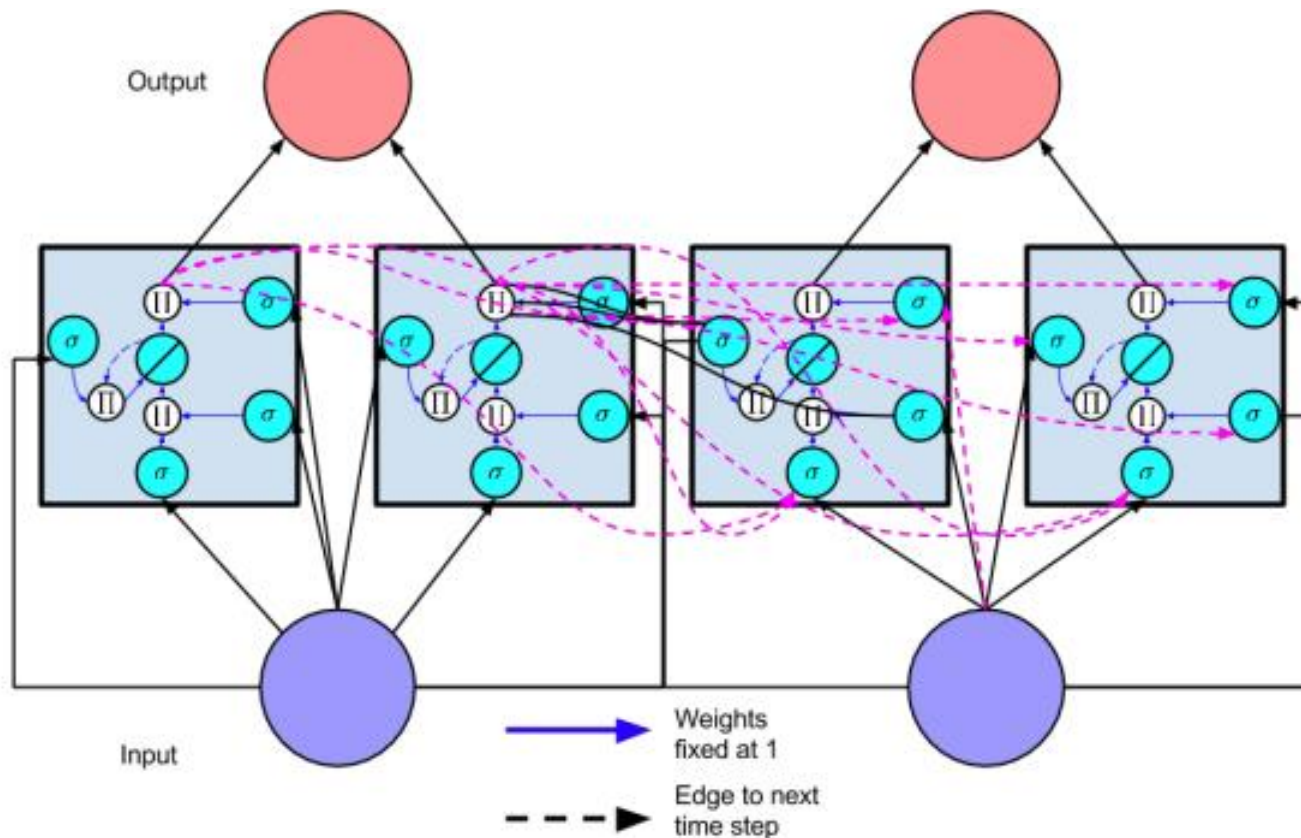
- Store input into memory
- Protect memory from overwritten by other irrelevant inputs

Forget gate



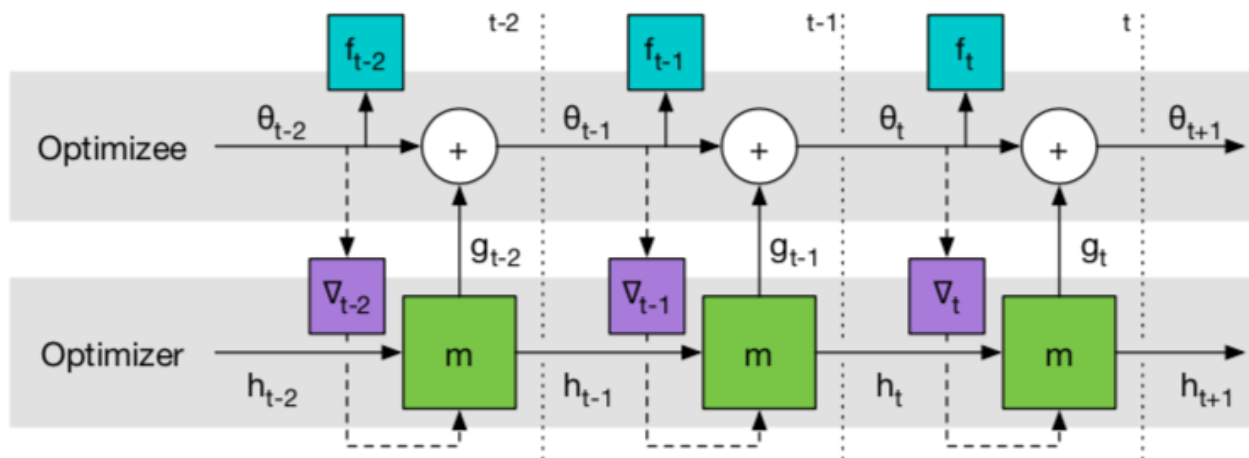
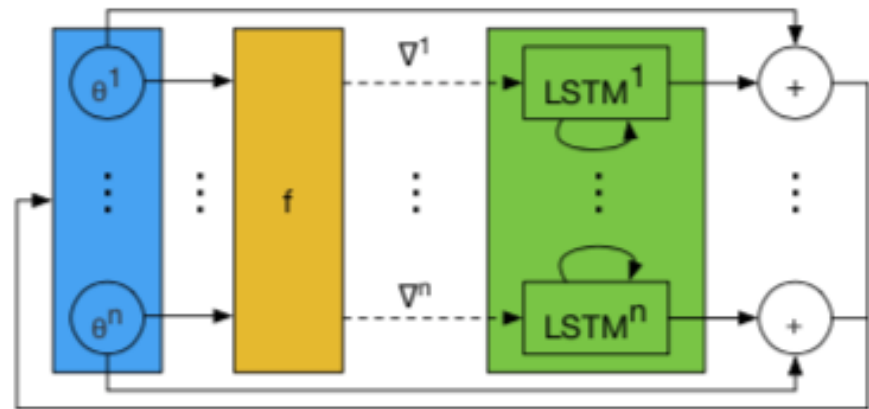
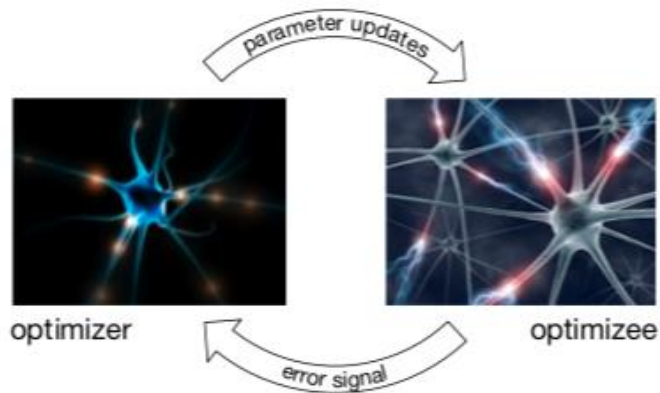
- Manipulates how much of previous cell state should be kept within the update to next state

LSTM Networks



Use LSTM memory cell as neurons to construct the network, e.g., replacing hidden layer neurons in MLP and establishing the recurrent relationship.

LSTM Application: learning to learn



$$\theta_{t+1} = \theta_t + g_t,$$

$$\begin{bmatrix} g_t \\ h_{t+1} \end{bmatrix} = m(\nabla_t, h_t, \phi)$$

Andrychowicz et al.,
NIPS 2016



Part IV: Self-Organizing Feature Map (SOFM) of Kohonen



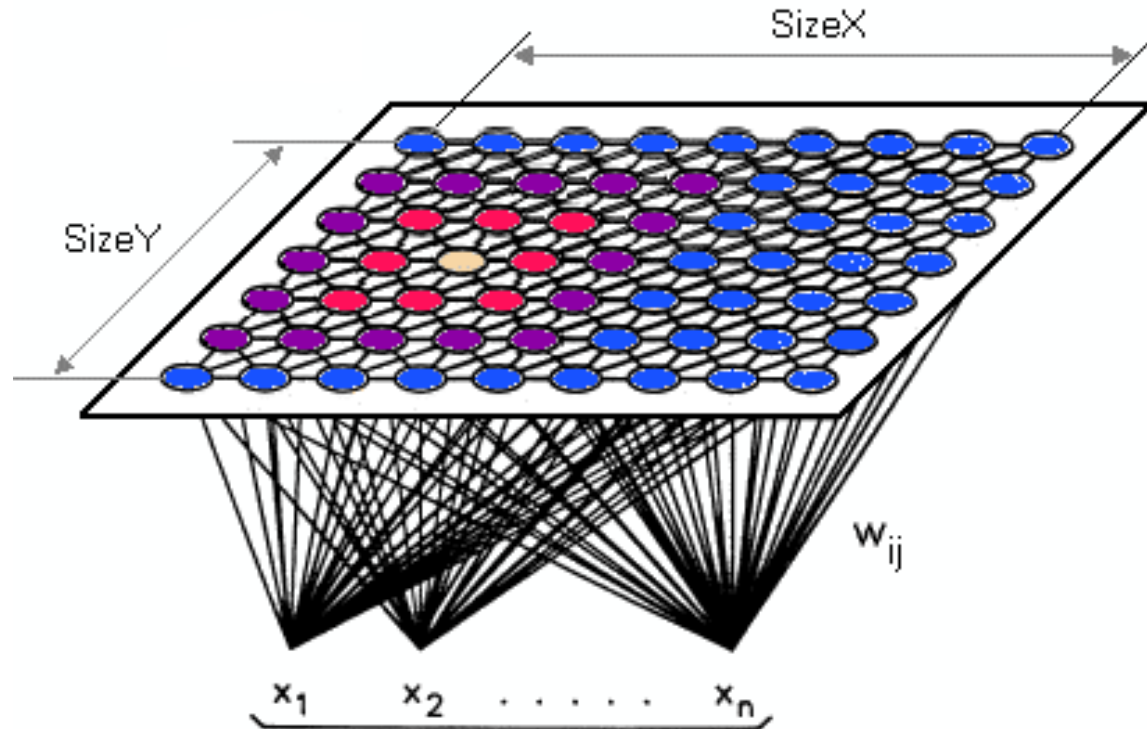


What is SOFM?

- ❖ Neural Network with **Unsupervised Learning**
- ❖ Dimensionality reduction concomitant with preservation of topological information.
- ❖ Three principals
 - Self-reinforcing
 - Competition
 - Cooperation



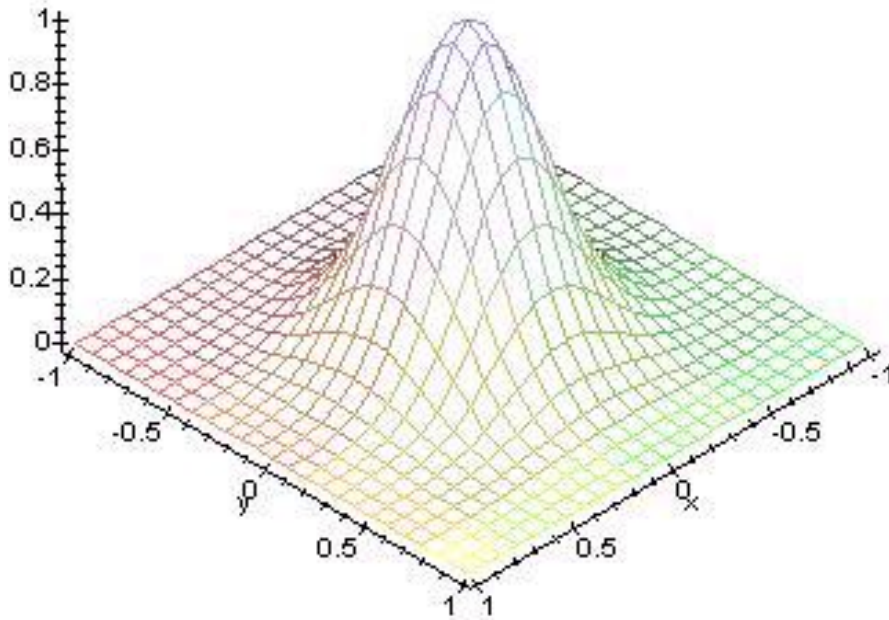
Structure of SOFM



Picture:

http://www.lohninger.com/helpcsuite/kohonen_network_-_background_information.htm

Mexican Hat Connectivity Pattern



Competition and Cooperation





Competition

- ❖ Finding the **best matching weight vector** for the present input.
- ❖ **Criterion** for determining the winning neuron:
Maximum Inner Product → Minimum Euclidean Distance





Cooperation

- ❖ Identify a **neighborhood** around the winning neuron.
- ❖ Topological neighborhood can be of different **shapes** such as **Square**, **Hexagonal**, or **Gaussian**.
- ❖ The **width** of the neighborhood is a function of time: as epochs of training elapse, the neighborhood **shrinks**.



Adaptation

- ❖ **Weights of neurons within the winning cluster are updated.**
- ❖ **Hebbian hypothesis** but with a forgetting term scaled by some functions of the neuronal signal.

$$w_j(t+1) = \begin{cases} w_j(t) + \eta(t)(x(t) - w_j(t)), & j \in \Lambda_i(t) \\ w_j(t), & j \notin \Lambda_i(t) \end{cases}$$



Operational Summary of SOFM algorithm

Repeat

{

Selection: Pick a sample

Similarity Matching: Find the winning neuron

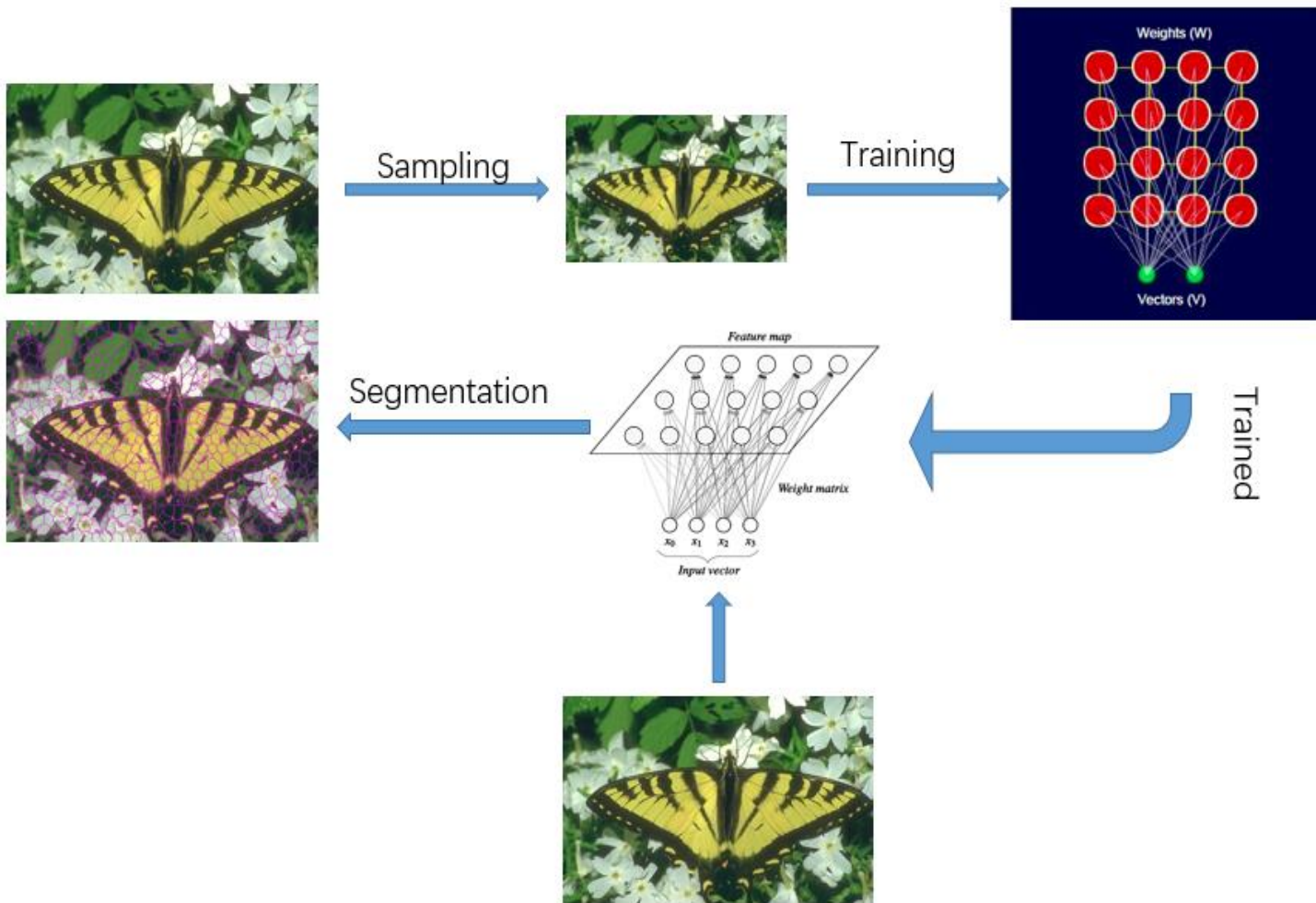
Adaptation: Update synaptic vectors of ONLY
the winning cluster.

Update: Update the learning rate and
neighborhood

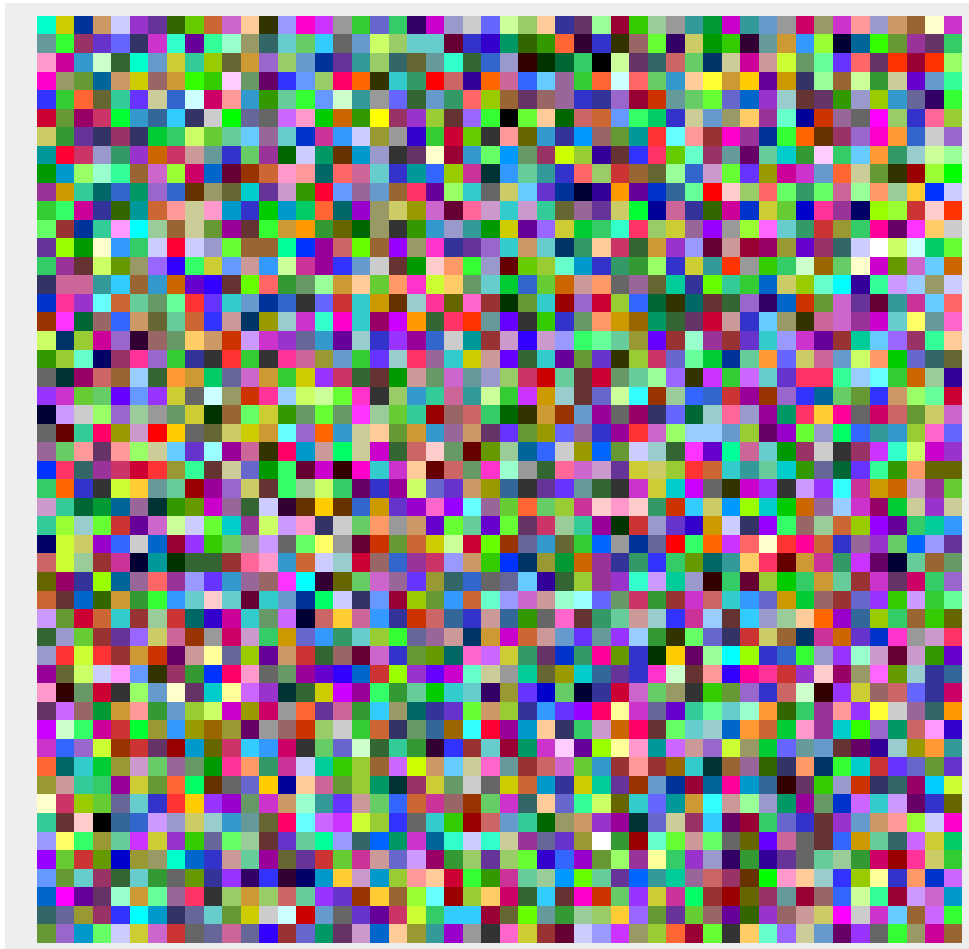
}Until (there is no observable change in the map)



SOFM Application: Superpixel Segmentation



SOFM Application: Color Clustering



Can you organize these colors in order, i.e., having similar colors close to each?



Can you understand artificial neural network and apply it to special applications?

LIUXIABI@BIT.EDU.CN

